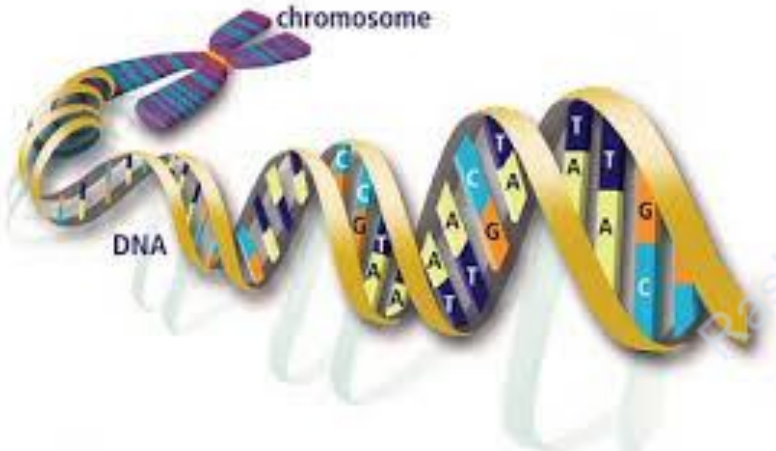
Sequence Modeling

Rasha Obeidat®, PhD

Deep learning

Sequential and Temporal data

- Certain types of data are serial in nature. E.g., text, speech, stock market, DNA, etc.



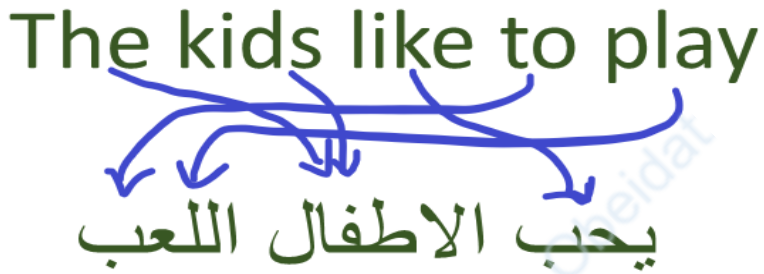
Atkinson briefly embarked in doctoral work before devoting his full attention to acting. First winning national attention in The Oxford Revue at the Edinburgh Festival Fringe in August 1976 he had already written and performed sketches for shows in Oxford by the Etceteras.

Examples of sequence modeling

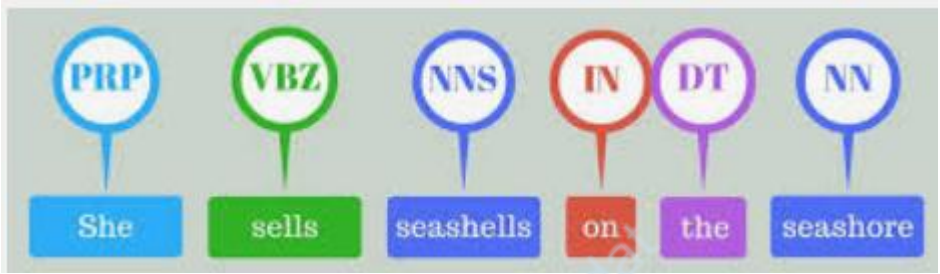
- Speech recognition:
- Machine Translation:
- Question Answering:
- DNA sequence Analysis:
- Entity Typing
- Image captioning.
- Music Generation:
- Stok market prediction:
- Sentiment Analysis:
- Video Activity recognition:
- Handwriting Recognition.

Examples of sequence modeling

- Machine Translation:



- Part-of-Speech tagging:



Examples of sequence modeling

- Speech recognition:

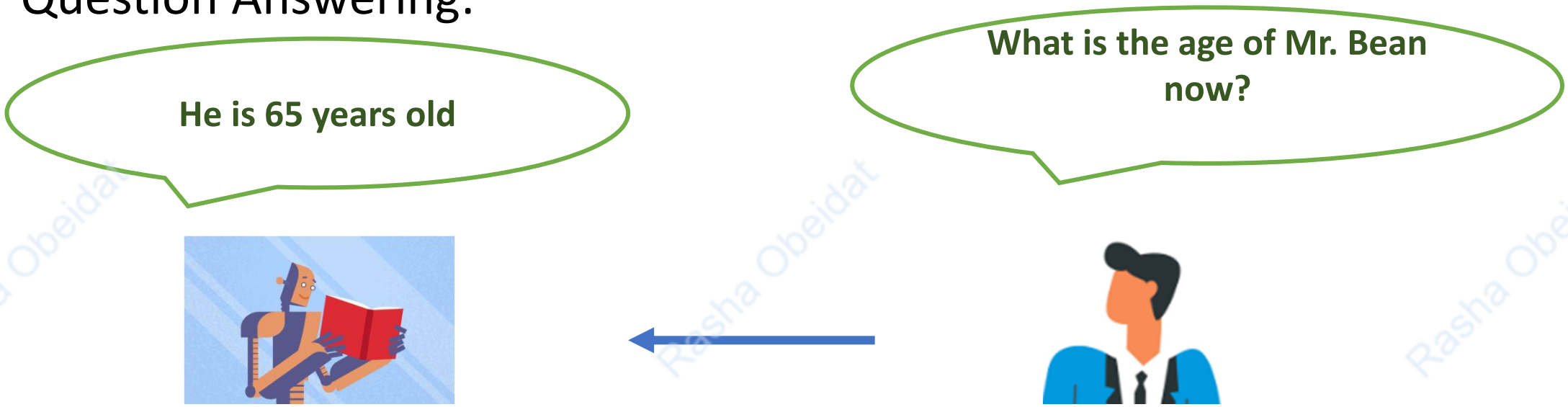
I need to book a flight from Amman to Dubai

- DNA sequence Analysis:

TTAGCCAGTCAGTCCCGGAT → TTAGCC**AGTCAGTCCCGG**AT

Examples of sequence modeling

- Question Answering:



- Entity Typing

The youngest of four brothers, his parents were **[Eric Atkinson]**(**person/Artist**) , a farmer and company director,

Examples of sequence modeling

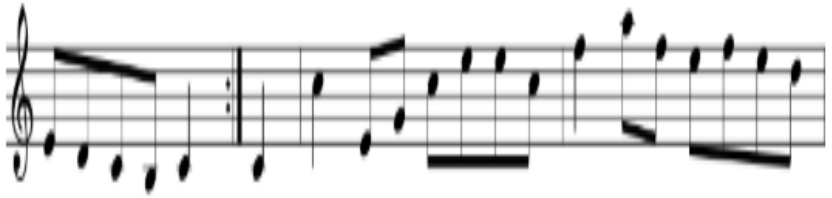
- Image captioning.

- Video Activity recognition:

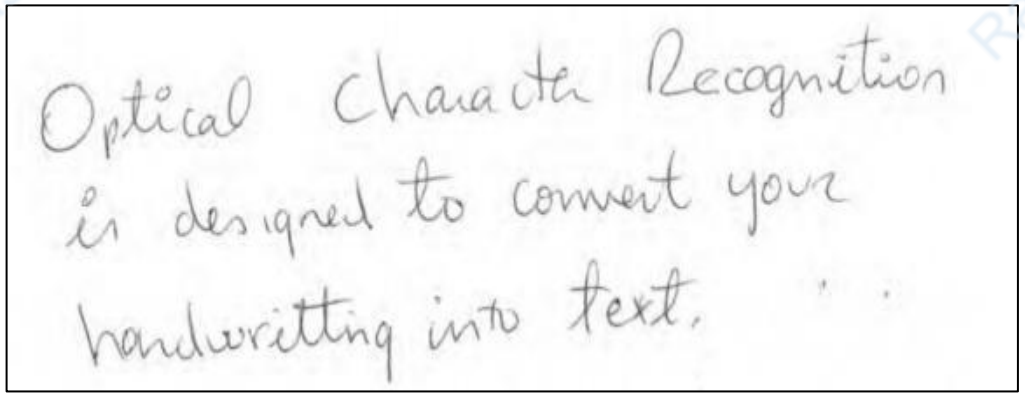
Dancing

Examples of sequence modeling

- Music Generation:

ø → 

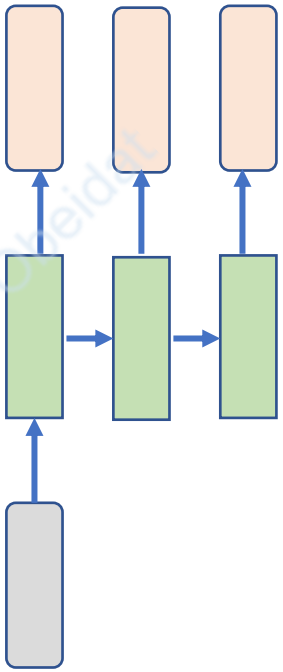
- Hand Writing Recognition.



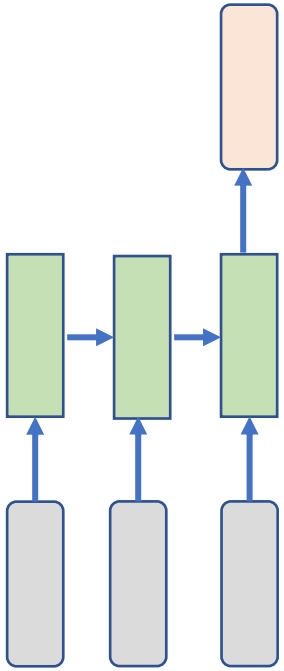
→ Optical Character Recognition is designed to convert your handwriting into text.

Nature of Sequence Data

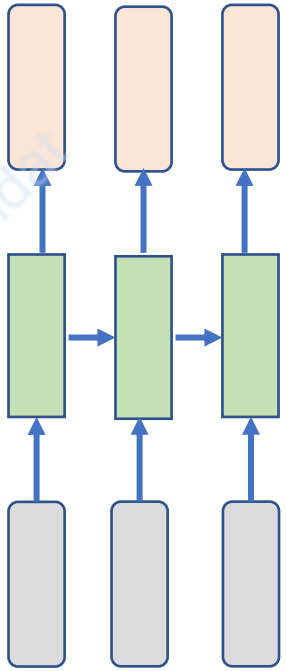
- Sequence Data can take different input/output shapes.



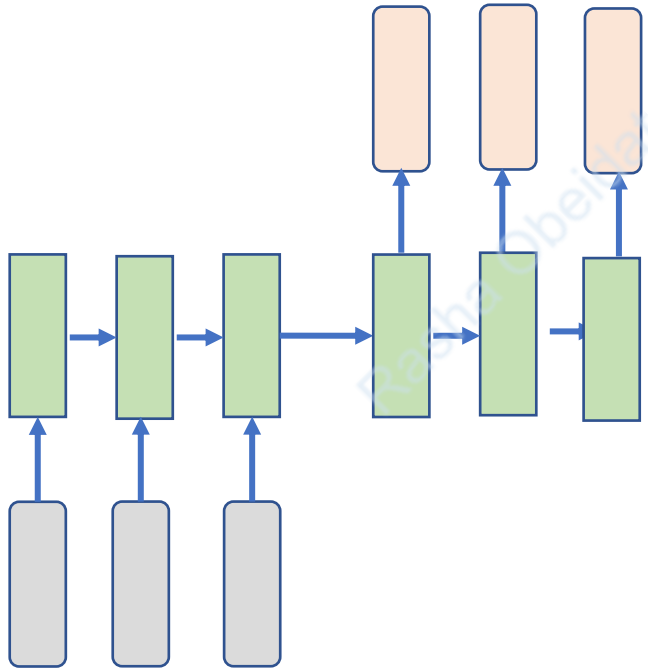
One-to-many
E.g., Image Captioning



Many-to-one
E.g., Entity Typing



Many-to-many
E.g., POS tagging

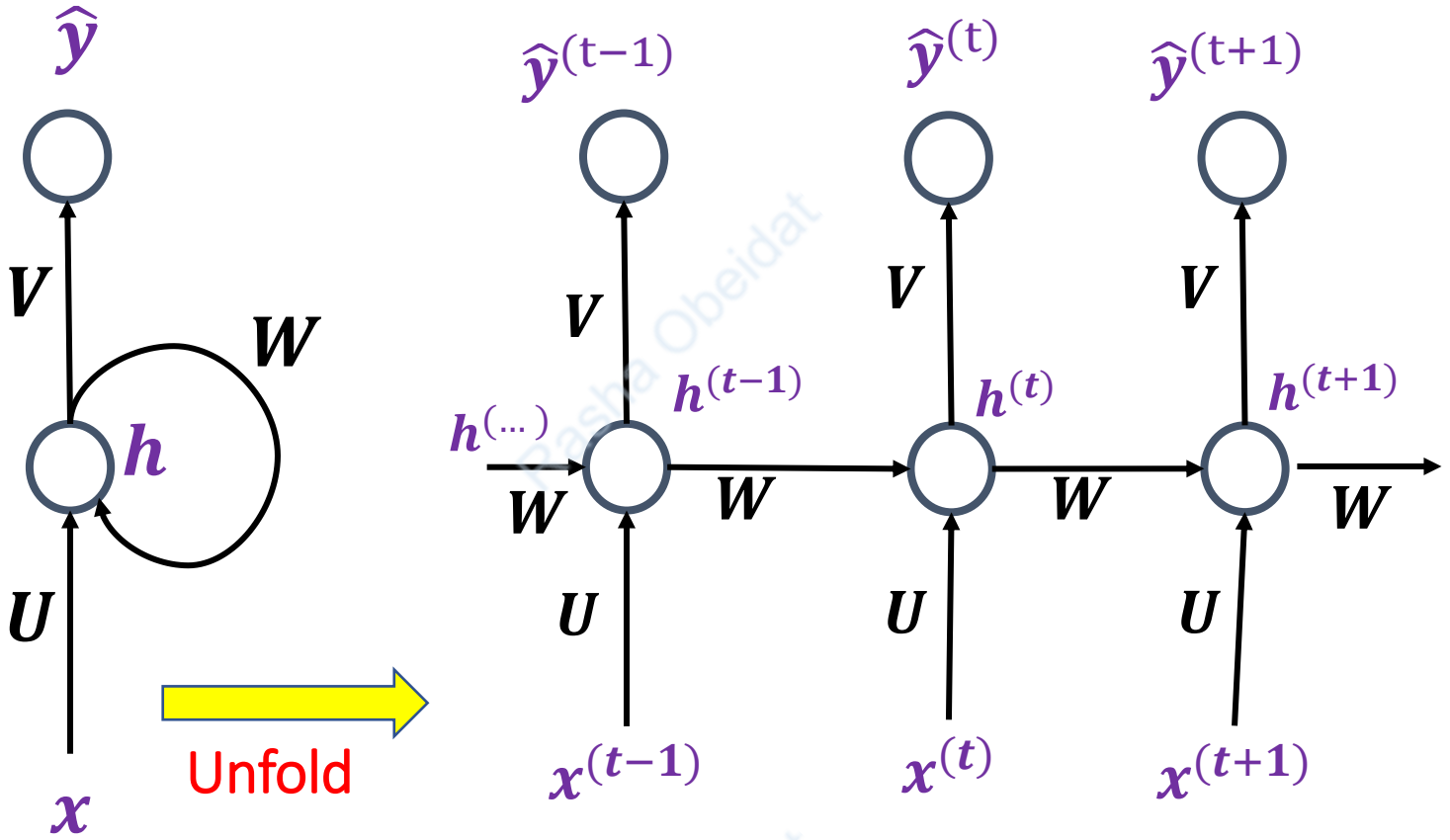


Many-to-many
E.g., Machine Translation

Why Recurrent Neural Networks?

- With traditional neural network we assume that all inputs (and outputs) are independent of each other.
- ***Recurrent Neural Networks (RNN)*** are family of neural networks used for processing sequential data.
- RNN better fit sequential and temporal data as It exhibit dynamic temporal behavior.
- RNNs are called ***recurrent*** because they perform the same task for every element of a sequence, with the output being depended on the previous computations.

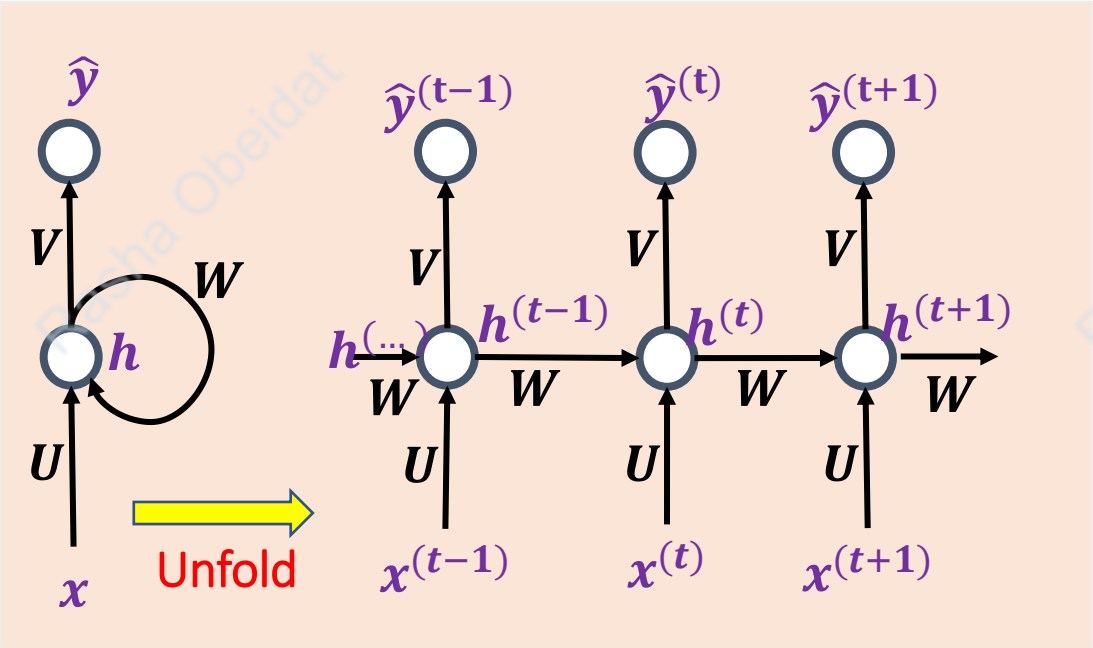
Vanilla RNN Graph



Vanilla RNN Graph

$\hat{y}^{(t)}$ can be considered as:

$$p(\hat{y}^{(t)}|x^1, x^2 \dots x^t; W, V, U)$$



$$\boldsymbol{x} = [\boldsymbol{x}^{(1)}, \dots \boldsymbol{x}^{(t-1)}, \boldsymbol{x}^{(t)}, \boldsymbol{x}^{(t+1)}, \dots \boldsymbol{x}^{(\tau)}]$$

$\boldsymbol{h}^{(t)}$ is the hidden state of time step t

- $a^{(t)} = Wh^{(t-1)} + Ux^{(t)} + b_i$
- $h^{(t)} = \tanh(a^{(t)})$
- $\hat{y}^{(t)} = \mathrm{g}(Vh^{(t)} + b_o)$

- $x \in \mathcal{R}^{\tau \times d}$: the input sequence representation.
  - $x^t \in \mathcal{R}^d$: input vector at time step t.
  - $\boldsymbol{U} \in \mathcal{R}^{m \times d}$: **input-to-hidden** weights
  - $\mathbf{W} \in \mathcal{R}^{m \times m}$: **hidden-to-hidden** weights
  - $\mathbf{V} \in \mathcal{R}^{k \times m}$: **hidden-to-output** weights

Where k is the number of classes, τ is the length of the input sequence. g is an activation function(e.g., $\sigma, softmax$).

Forward pass in RNN is a recurrent operation

- Suppose that $\tau = 3$

❑ $t = 1$

$h^1 = \tanh(Wh^0 + Ux^1 + b_i)$

$\hat{y}^1 = \mathrm{g}(Vh^1 + b_o)$

❑ $t = 2$

$h^2 = \tanh(Wh^1 + Ux^2 + b_i)$

$= \tanh(W \tanh(Wh^0 + Ux^1 + b_i) + Ux^2 + b_i)$

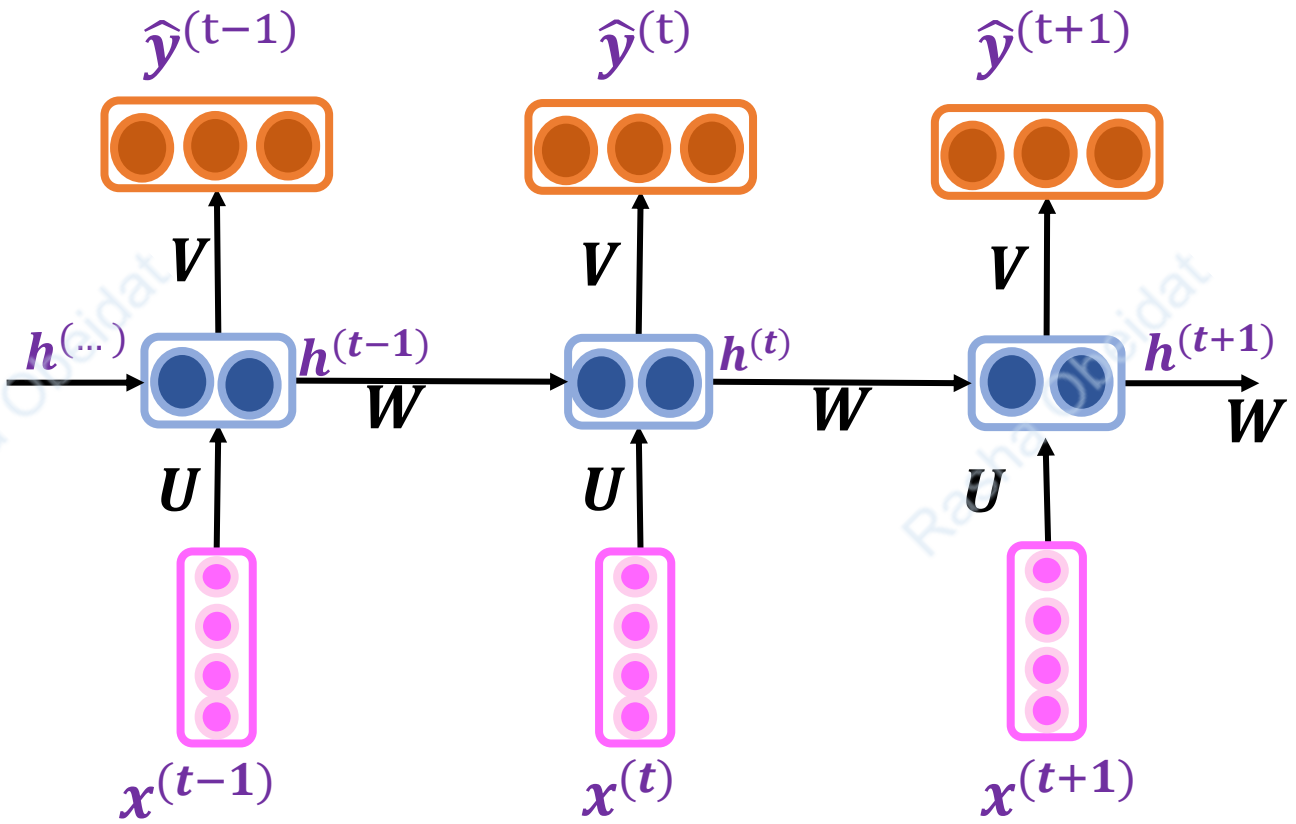
$\hat{y}^2 = \mathrm{g}(Vh^2 + b_o)$

❑ $t = 3$

$h^3 = \tanh(Wh^2 + Ux^3 + b_i)$

$= \tanh(W \tanh(W \tanh(Wh^0 + Ux^1 + b_i) + Ux^2 + b_i) + Ux^3 + b_i)$

$\hat{y}^3 = \mathrm{g}(Vh^3 + b_o)$



- $h^{(t)} = \tanh(Wh^{(t-1)} + Ux^{(t)} + b_i)$
- $\hat{y}^{(t)} = \mathrm{g}(Vh^{(t)} + b_o)$

Parameter sharing in RNN

- RNN shares the same weights across several time steps.
- ***Parameter sharing*** is important because it:
  1. Reduces the complexity of parameters of RNN which can be estimated with far fewer training examples than would be required without parameter sharing.

     Assume that we have different W, U, V for every time step!!

  2. Makes RNN able scale to long sequences with variable lengths and generalize to sequence lengths that did not appear in the training set.

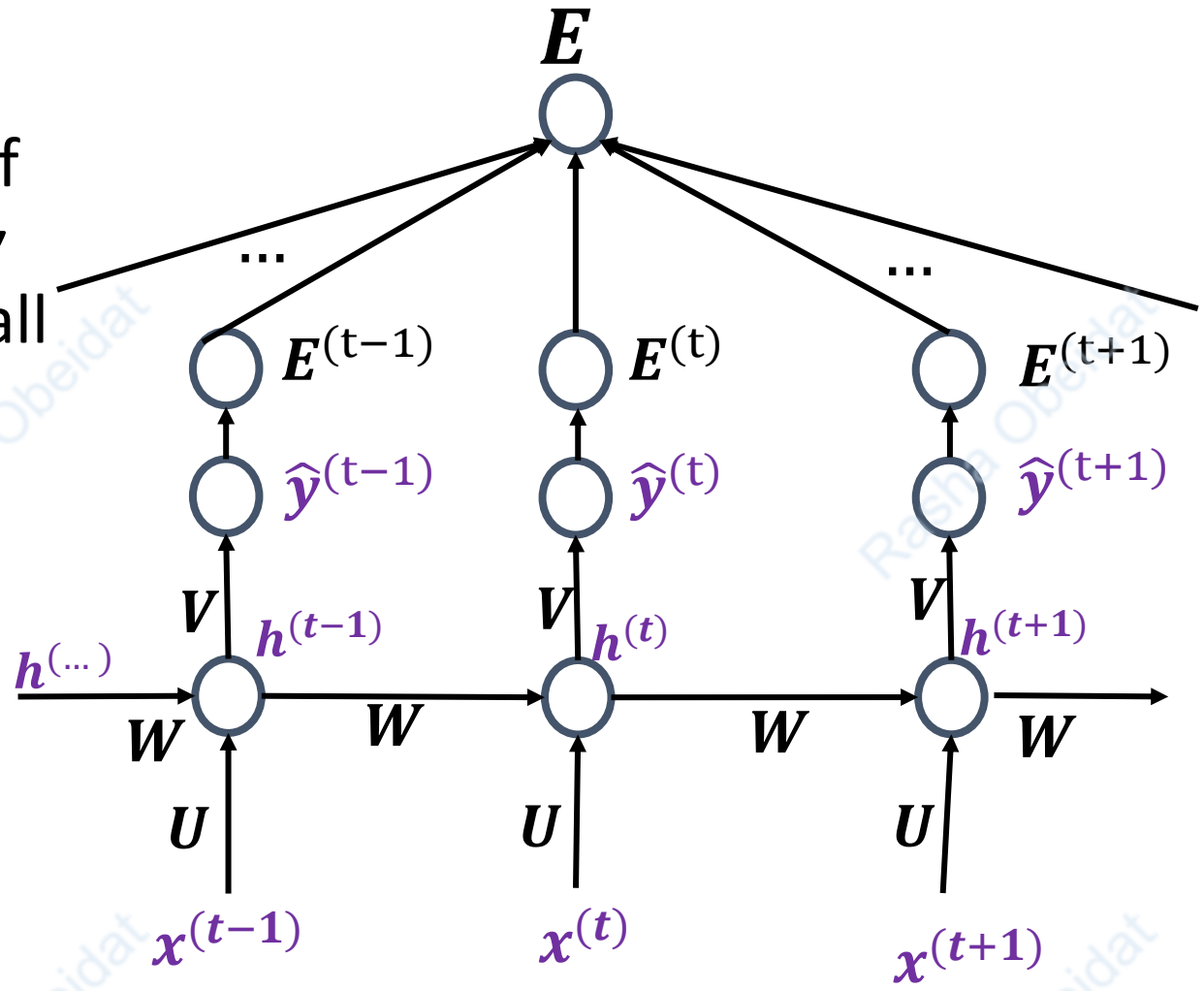
     Assume that that at testing time, we've got a sequence longer than all sequences encountered during the training!!

RNN Training.

- The total loss for a given sequence of x values paired with a sequence of y values is the sum of the losses over all the time steps.

$$E = E^{(1)} + \cdots + E^{(t)} + \cdots + E^{(\tau)}$$

- RNN are trained using ***back-propagation through time.***



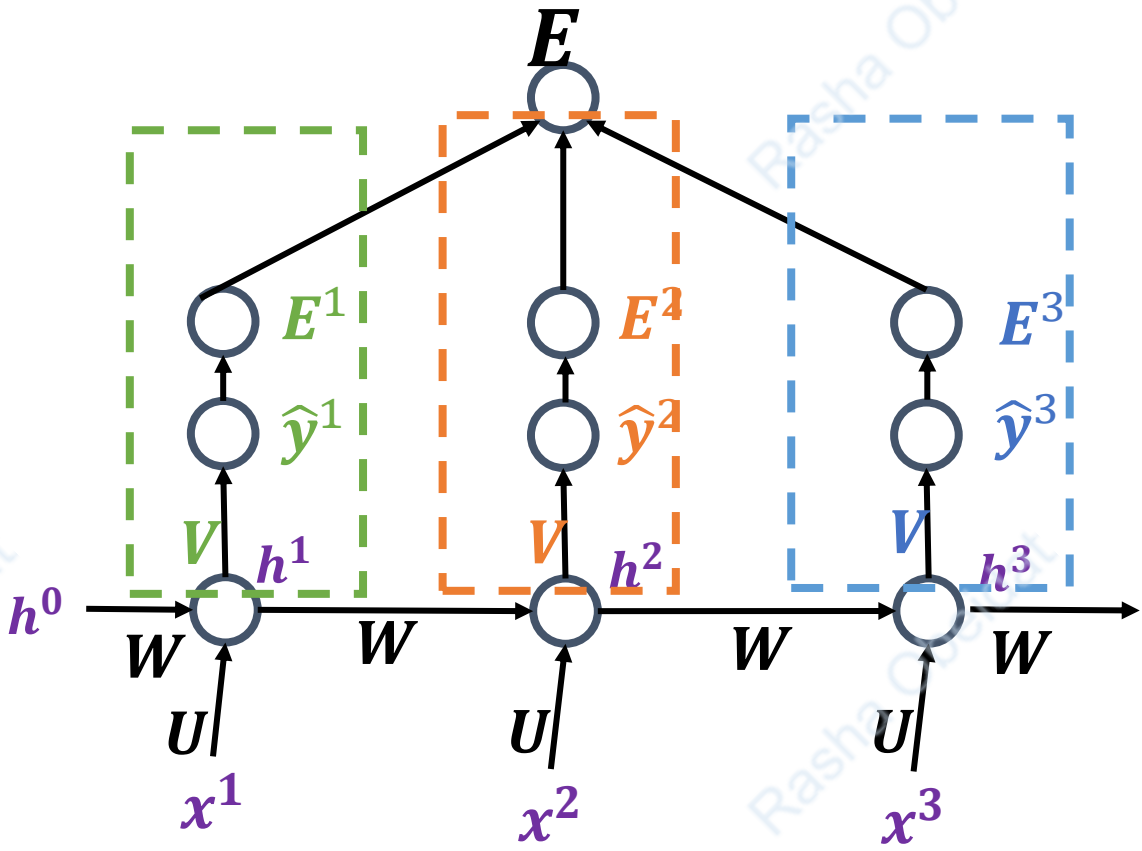
RNN Training.

Derivative with respect to V

$$\frac{\partial E}{\partial V} = \sum_{t=1}^{\tau} \frac{\partial E^t}{\partial \hat{y}^t} \frac{\partial \hat{y}^t}{\partial V}$$

- **E.g., for RNN with $\tau = 3$**

$$\frac{\partial E}{\partial V} = \frac{\partial E^1}{\partial \hat{y}^1}\frac{\partial \hat{y}^1}{\partial V} + \frac{\partial E^2}{\partial \hat{y}^2}\frac{\partial \hat{y}^2}{\partial V} + \frac{\partial E^3}{\partial \hat{y}^3}\frac{\partial \hat{y}^3}{\partial V}$$



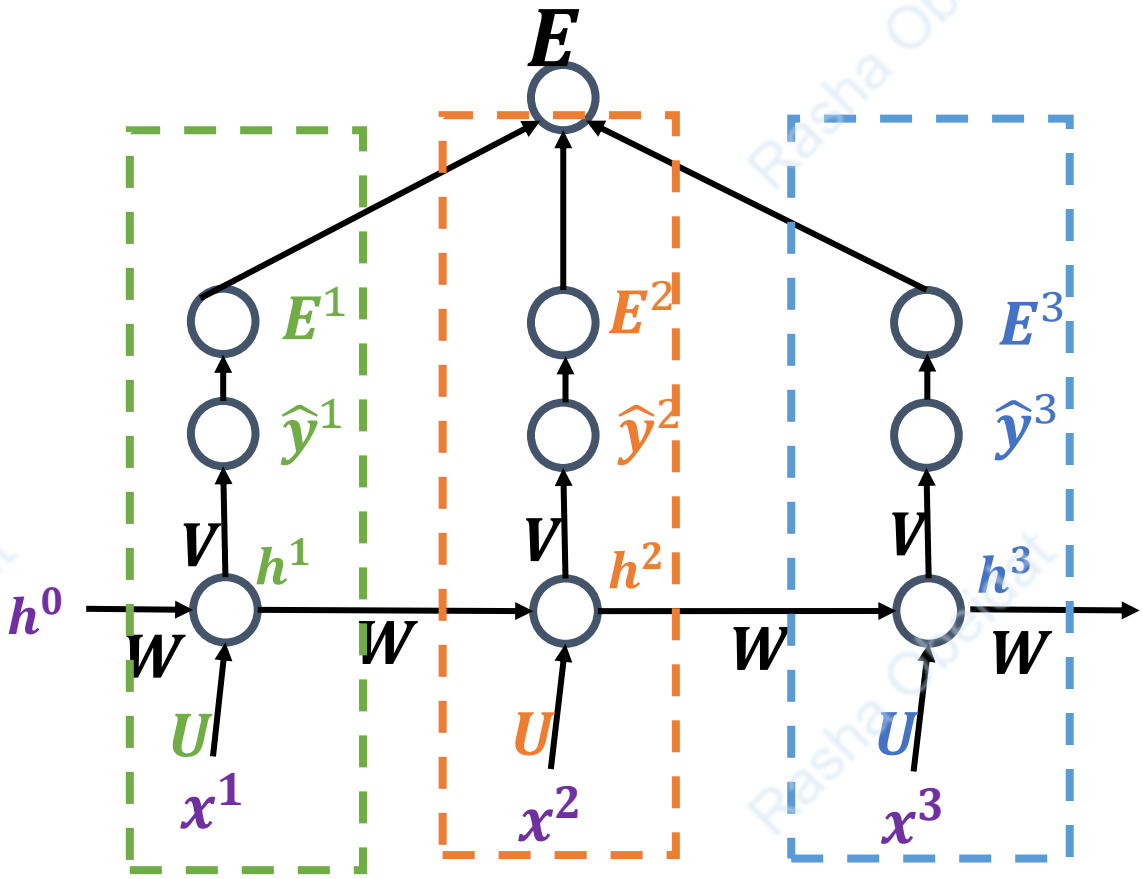
$$a^{(t)} = Wh^{(t-1)} + Ux^{(t)} + b_i$$
$$h^{(t)} = tanh(a^{(t)})$$
$$\hat{y}^{(t)} = g(Vh^{(t)} + b_o)$$

RNN Training.

Derivative with respect to V

$$\frac{\partial E}{\partial U} = \sum_{t=1}^{\tau} \frac{\partial E^t}{\partial \hat{y}^t} \frac{\partial \hat{y}^t}{\partial h^t} \frac{\partial h^t}{\partial a^t} \frac{\partial a^t}{\partial U}$$

E.g., for RNN with $\tau = 3$



$$\frac{\partial E}{\partial U} = \frac{\partial E^1}{\partial U} + \frac{\partial E^2}{\partial U} + \frac{\partial E^3}{\partial U} =$$

$$a^{(t)} = Wh^{(t-1)} + Ux^{(t)} + b_i$$
$$h^{(t)} = tanh(a^{(t)})$$
$$\hat{y}^{(t)} = g(Vh^{(t)} + b_o)$$

$$\frac{\partial E^1}{\partial \hat{y}^1} \frac{\partial \hat{y}^1}{\partial h^1} \frac{\partial h^1}{\partial a^1} \frac{\partial a^1}{\partial U} + \frac{\partial E^2}{\partial \hat{y}^2} \frac{\partial \hat{y}^2}{\partial h^2} \frac{\partial h^2}{\partial a^2} \frac{\partial a^2}{\partial U} + \frac{\partial E^3}{\partial \hat{y}^3} \frac{\partial \hat{y}^3}{\partial h^3} \frac{\partial h^3}{\partial a^3} \frac{\partial a^3}{\partial U}$$

RNN Training.

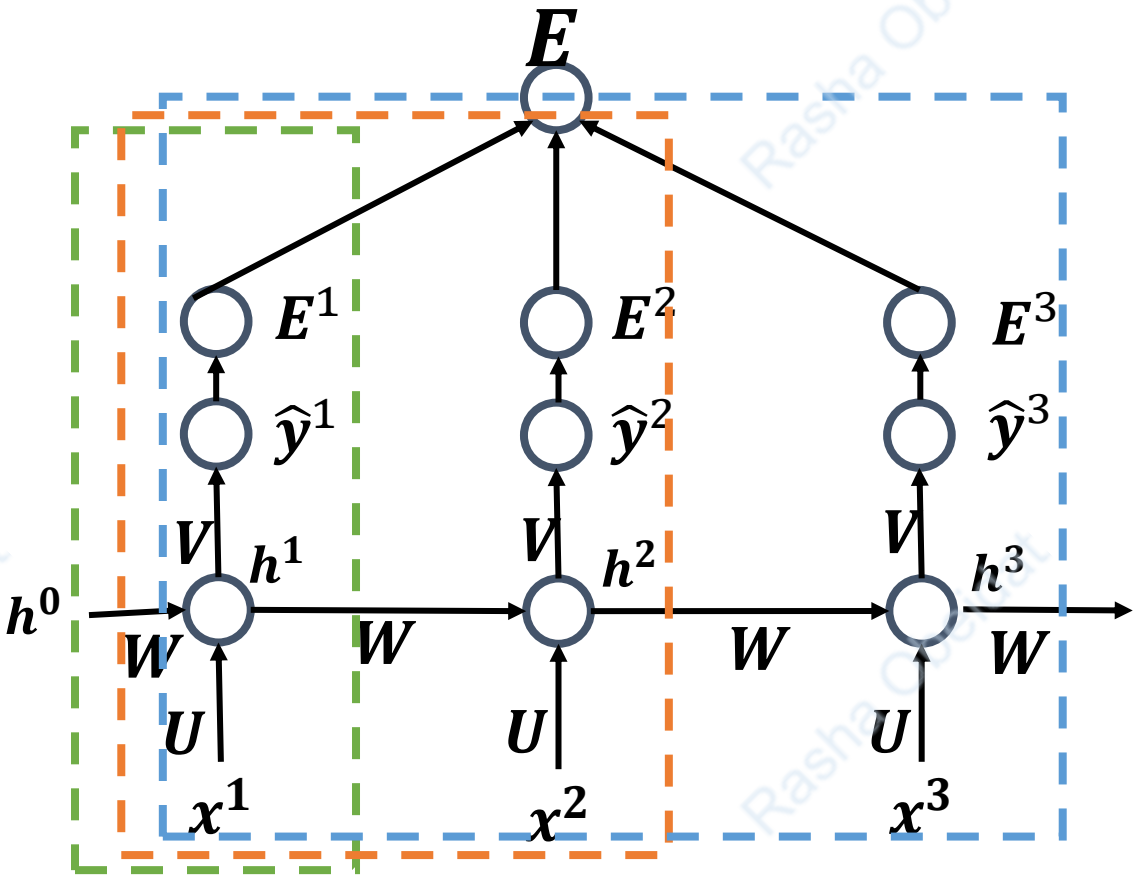
Derivative of E with respect to W

$$\frac{\partial E}{\partial W} = \sum_{t=1}^{\tau} \frac{\partial E^t}{\partial W}$$

$$\frac{\partial E^t}{\partial W} = \sum_{k=1}^{t} \frac{\partial E^t}{\partial \hat{y}^t} \frac{\partial \hat{y}^t}{\partial h^t} \frac{\partial h^t}{\partial h^k} \frac{\partial h^k}{\partial W}$$

For example, for $\tau = 3$:

$$\frac{\partial E}{\partial W} = \frac{\partial E^1}{\partial W} + \frac{\partial E^2}{\partial W} + \frac{\partial E^3}{\partial W}$$



$$h^{(t)} = tanh(Wh^{(t-1)} + Ux^{(t)} + b_i)$$
$$\hat{y}^{(t)} = g(Vh^{(t)} + b_o)$$

RNN Training.

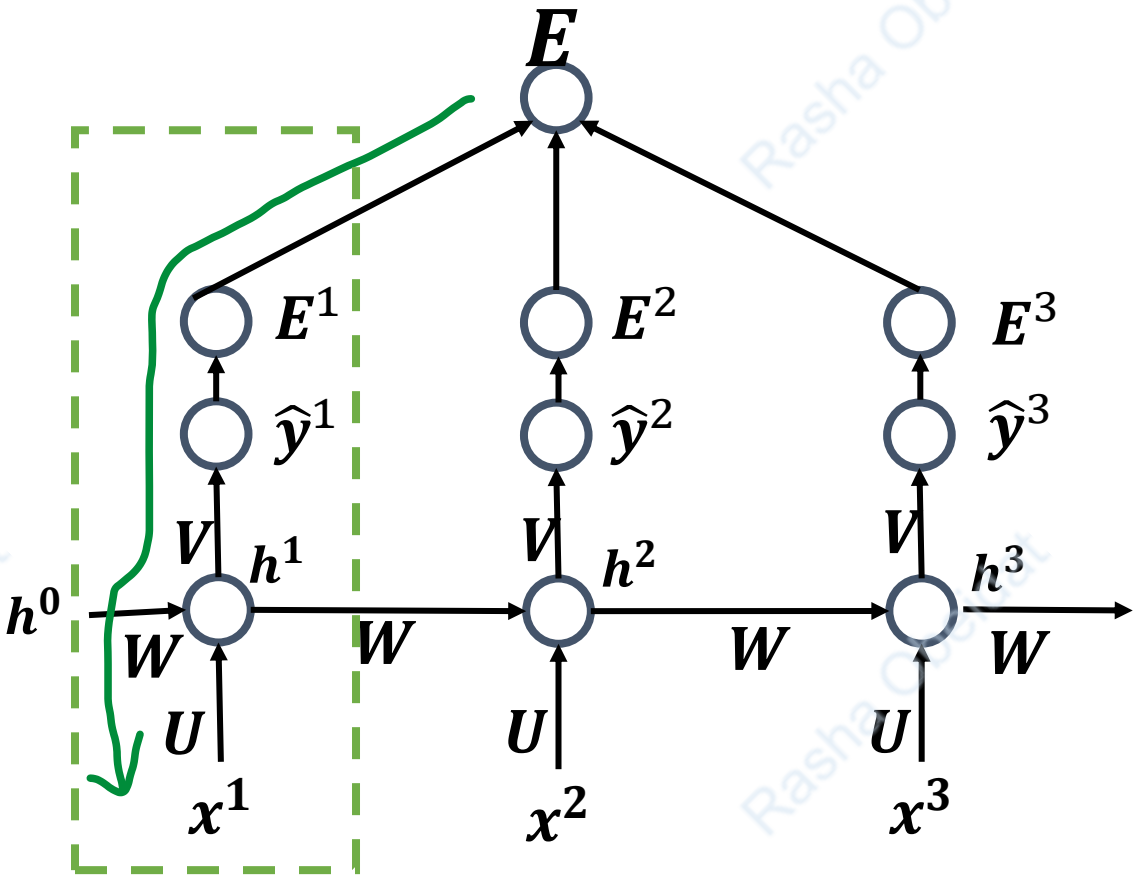
Derivative of E with respect to W

$$\frac{\partial E}{\partial W} = \sum_{t=1}^{\tau} \frac{\partial E^t}{\partial W}$$

$$\frac{\partial E^t}{\partial W} = \sum_{k=1}^{t} \frac{\partial E^t}{\partial \hat{y}^t} \frac{\partial \hat{y}^t}{\partial h^t} \frac{\partial h^t}{\partial h^k} \frac{\partial h^k}{\partial W}$$

For example, for $\tau = 3$:

$$\frac{\partial E^1}{\partial W} = \frac{\partial E^1}{\partial \hat{y}^1} \frac{\partial \hat{y}^1}{\partial h^1} \frac{\partial h^1}{\partial W}$$



$$h^{(t)} = tanh(Wh^{(t-1)} + Ux^{(t)} + b_i)$$
$$\hat{y}^{(t)} = g(Vh^{(t)} + b_o)$$

RNN Training.

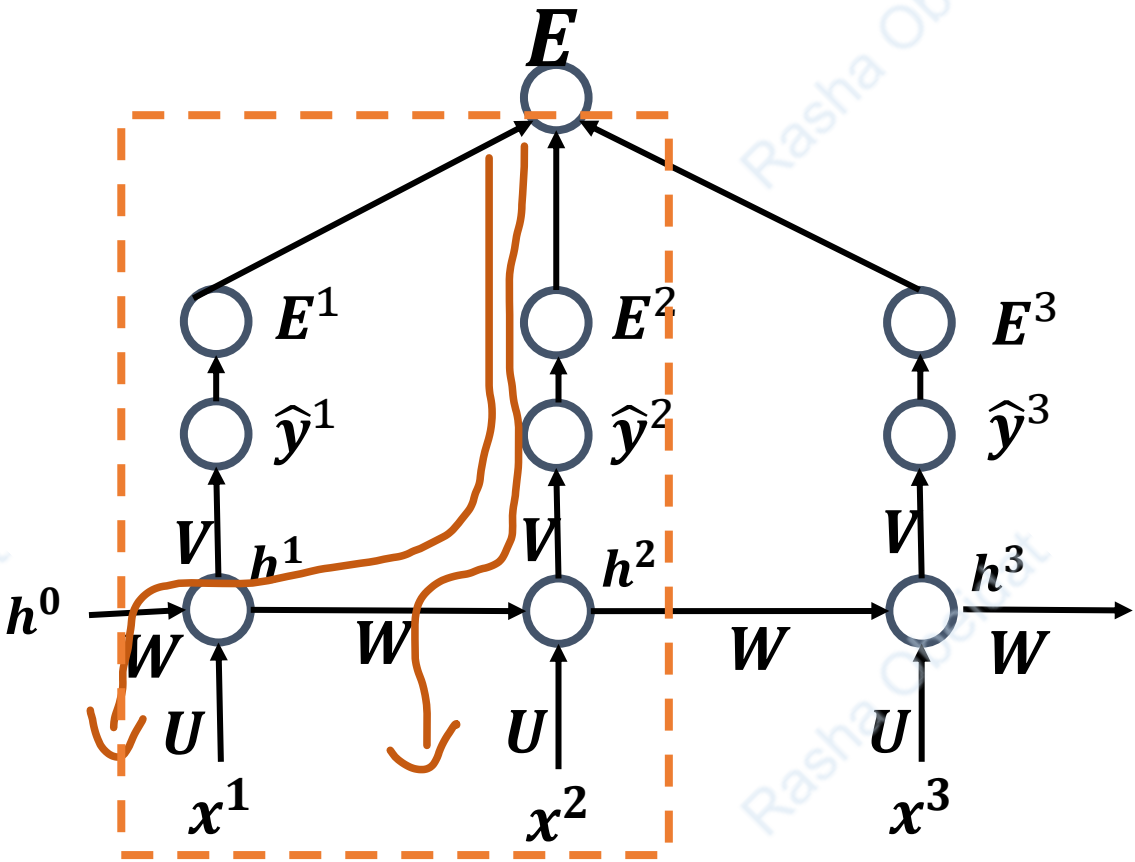
Derivative of E with respect to W

$$\frac{\partial E}{\partial W} = \sum_{t=1}^{\tau} \frac{\partial E^t}{\partial W}$$

$$\frac{\partial E^t}{\partial W} = \sum_{k=1}^{t} \frac{\partial E^t}{\partial \hat{y}^t} \frac{\partial \hat{y}^t}{\partial h^t} \frac{\partial h^t}{\partial h^k} \frac{\partial h^k}{\partial W}$$

For example, for $\tau = 3$:

$$\frac{\partial E^2}{\partial W} = \frac{\partial E^2}{\partial \hat{y}^2} \frac{\partial \hat{y}^2}{\partial h^2} \frac{\partial h^2}{\partial h^1} \frac{\partial h^1}{\partial W} + \frac{\partial E^2}{\partial \hat{y}^2} \frac{\partial \hat{y}^2}{\partial h^2} \frac{\partial h^2}{\partial W}$$



$$h^{(t)} = tanh(Wh^{(t-1)} + Ux^{(t)} + b_i)$$
$$\hat{y}^{(t)} = g(Vh^{(t)} + b_o)$$

RNN Training.

Derivative of E with respect to W

$$\frac{\partial E}{\partial W} = \sum_{t=1}^{\tau} \frac{\partial E^t}{\partial W}$$

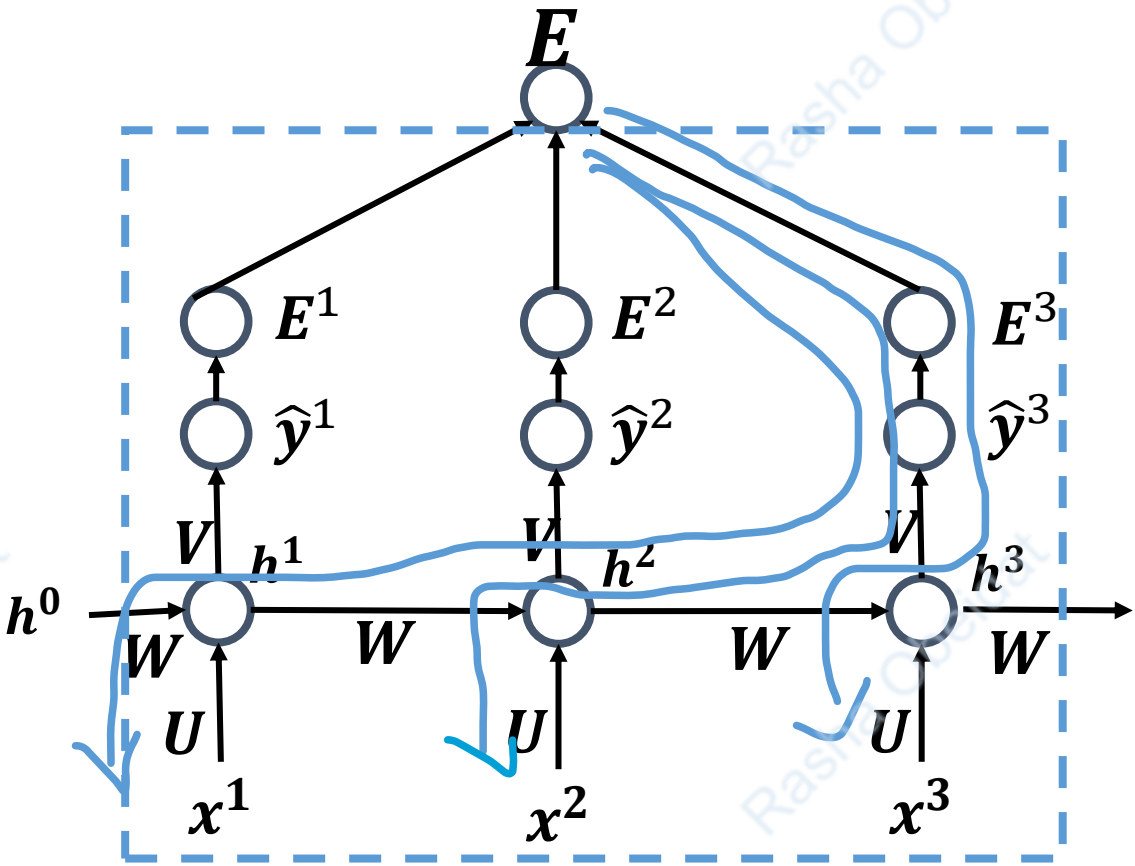
$$\frac{\partial E^t}{\partial W} = \sum_{k=1}^{t} \frac{\partial E^t}{\partial \hat{y}^t} \frac{\partial \hat{y}^t}{\partial h^t} \frac{\partial h^t}{\partial h^k} \frac{\partial h^k}{\partial W}$$

For example, for $\tau = 3$:

$$\frac{\partial E^3}{\partial W} = \frac{\partial E^3}{\partial \hat{y}^3} \frac{\partial \hat{y}^3}{\partial h^3} \frac{\partial h^3}{\partial h^1} \frac{\partial h^1}{\partial W} + \frac{\partial E^3}{\partial \hat{y}^3} \frac{\partial \hat{y}^3}{\partial h^3} \frac{\partial h^3}{\partial h^2} \frac{\partial h^2}{\partial W} + \frac{\partial E^3}{\partial \hat{y}^3} \frac{\partial \hat{y}^3}{\partial h^3} \frac{\partial h^3}{\partial W}$$

Because (with ignoring the bias):

$$h^3 = tanh(Wh^2 + Ux^3)$$
$$= tanh(W\, tanh(Wh^1 + Ux^2) + Ux^3)$$



$$h^{(t)} = tanh(Wh^{(t-1)} + Ux^{(t)} + b_i)$$
$$\hat{y}^{(t)} = g(Vh^{(t)} + b_o)$$

RNN Training.

Derivative of E with respect to W

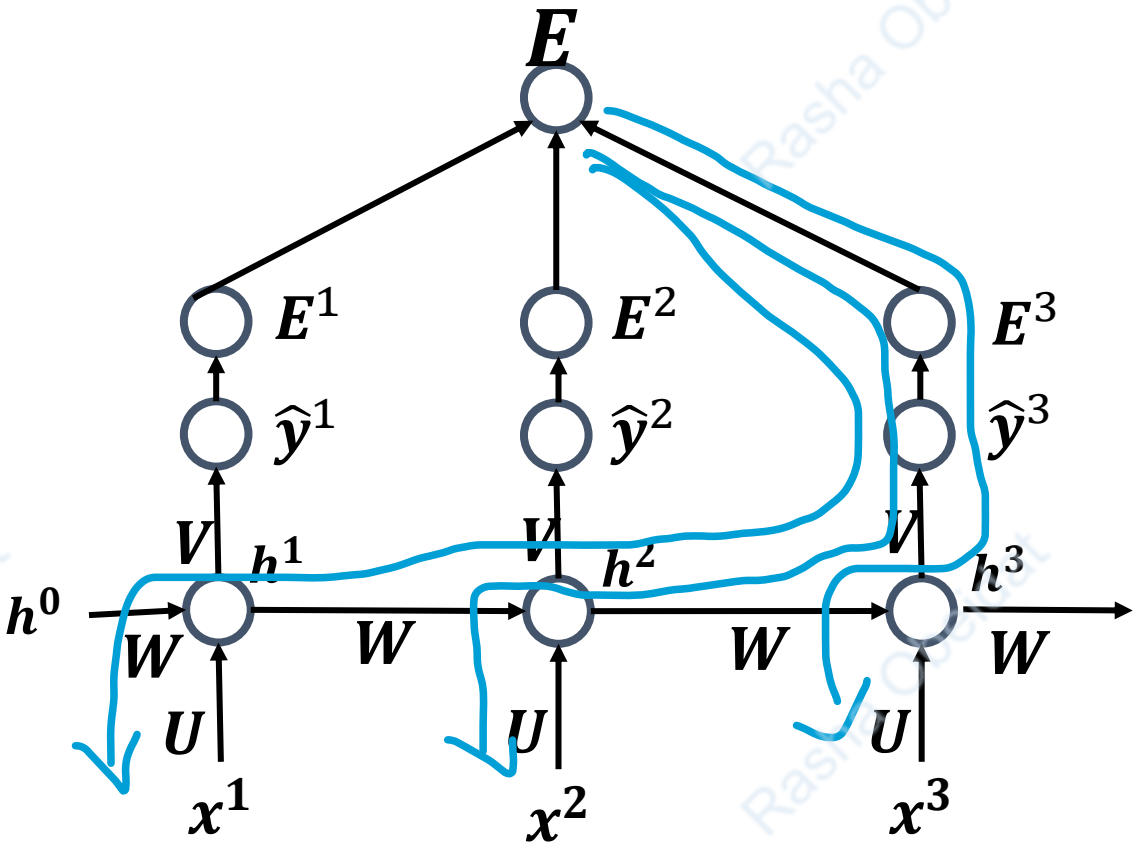
$$\frac{\partial E}{\partial W} = \sum_{t=1}^{\tau} \frac{\partial E^t}{\partial W}$$

$$\frac{\partial E^t}{\partial W} = \sum_{k=1}^{t} \frac{\partial E^t}{\partial \hat{y}^t} \frac{\partial \hat{y}^t}{\partial h^t} \frac{\partial h^t}{\partial h^k} \frac{\partial h^k}{\partial W}$$

For example, for $\tau = 3$:

$$\frac{\partial E^3}{\partial W} = \frac{\partial E^3}{\partial \hat{y}^3} \frac{\partial \hat{y}^3}{\partial h^3} \frac{\partial h^3}{\partial h^1} \frac{\partial h^1}{\partial w} + \frac{\partial E^3}{\partial \hat{y}^3} \frac{\partial \hat{y}^3}{\partial h^3} \frac{\partial h^3}{\partial h^2} \frac{\partial h^2}{\partial w} + \frac{\partial E^3}{\partial \hat{y}^3} \frac{\partial \hat{y}^3}{\partial h^3} \frac{\partial h^3}{\partial W}$$

$$\frac{\partial h^3}{\partial h^1} = \frac{\partial h^3}{\partial h^2} \frac{\partial h^2}{\partial h^1}$$



$$h^{(t)} = tanh(Wh^{(t-1)} + Ux^{(t)} + b_i)$$
$$\hat{y}^{(t)} = g(Vh^{(t)} + b_o)$$

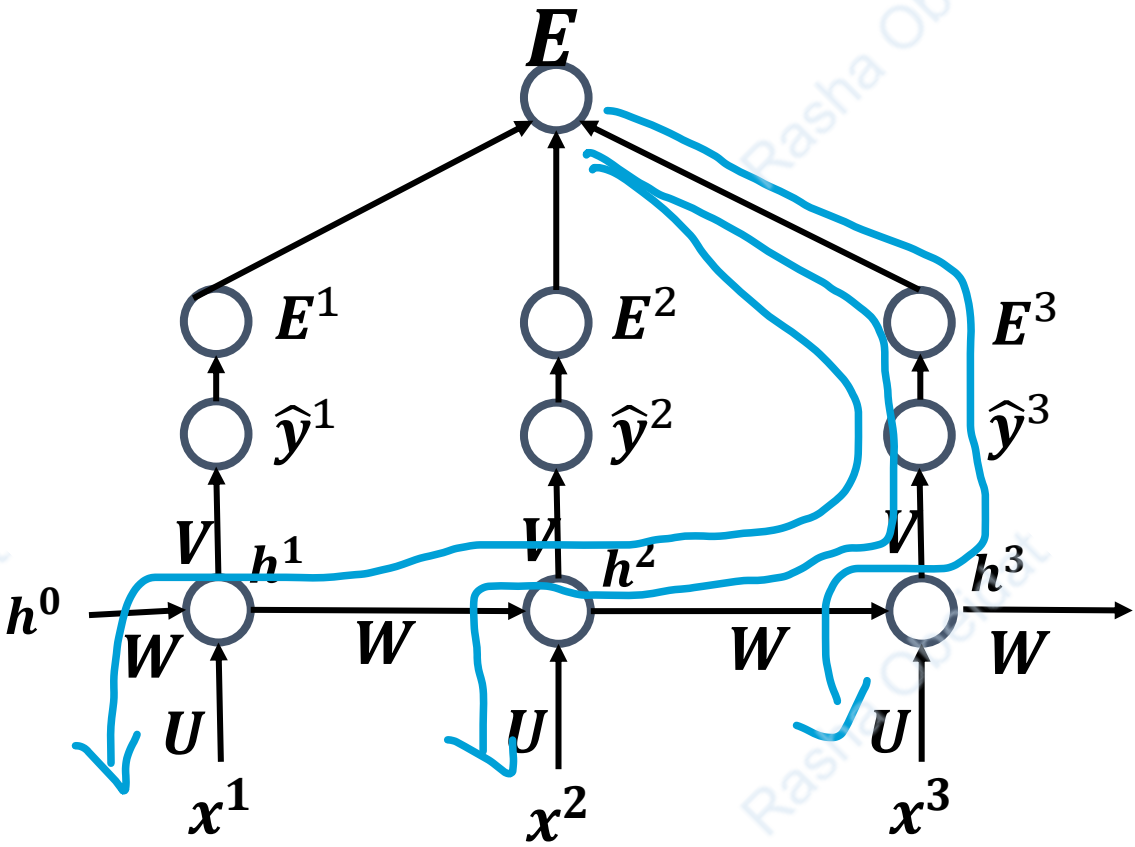
RNN Training.

Derivative of E with respect to W

$$\frac{\partial E}{\partial W} = \sum_{t=1}^{\tau} \frac{\partial E^t}{\partial W}$$

$$\frac{\partial E^t}{\partial W} = \sum_{k=1}^{t} \frac{\partial E^t}{\partial \hat{y}^t} \frac{\partial \hat{y}^t}{\partial h^t} \frac{\partial h^t}{\partial h^k} \frac{\partial h^k}{\partial W}$$

$$= \sum_{k=1}^{t} \frac{\partial E^t}{\partial \hat{y}^t} \frac{\partial \hat{y}^t}{\partial h^t} \frac{\partial h^t}{\partial h^{t-1}} \frac{\partial h^{t-1}}{\partial h^{t-2}} \cdots \frac{\partial h^{k+1}}{\partial h^k} \frac{\partial h^k}{\partial W}$$



$$h^{(t)} = tanh(Wh^{(t-1)} + Ux^{(t)} + b_i)$$
$$\hat{y}^{(t)} = g(Vh^{(t)} + b_o)$$

What is the problem with $\frac{\partial h^t}{\partial h^k}$?

- $\frac{\partial h^t}{\partial h^k} = \frac{\partial h^t}{\partial h^{t-1}} \frac{\partial h^{t-1}}{\partial h^{t-2}} \dots \frac{\partial h^{k+1}}{\partial h^k}$
- $\frac{\partial h^{t-1}}{\partial h^{t-2}} = \dots = \frac{\partial h^{k+1}}{\partial h^k} = \acute{tanh}(a)W$
- $\frac{\partial h^t}{\partial h^k} = \prod_{j=k}^{t} \frac{\partial h^{j+1}}{\partial h^j} W$
- This means that there is W^r where $r = t - k + 1$ in the gradient. What is wrong with W^k

$\acute{tanh}(a)$ is the derivative of the activation function $tanh(a)$, which is bounded between 0 and 1

Vanishing and exploding gradient

- Assume that W^k is a diagonalizable matrix, then:

$$W = PDP^{-1} \quad \Rightarrow \quad W^r = PD^rP^{-1}$$

D is the diagonal matrix of eigen values:

What D has eigen values = 5 for example and suppose that $r = 4, \Rightarrow D^{\mathrm{k}}$ will have 625 → (Exploding gradient)

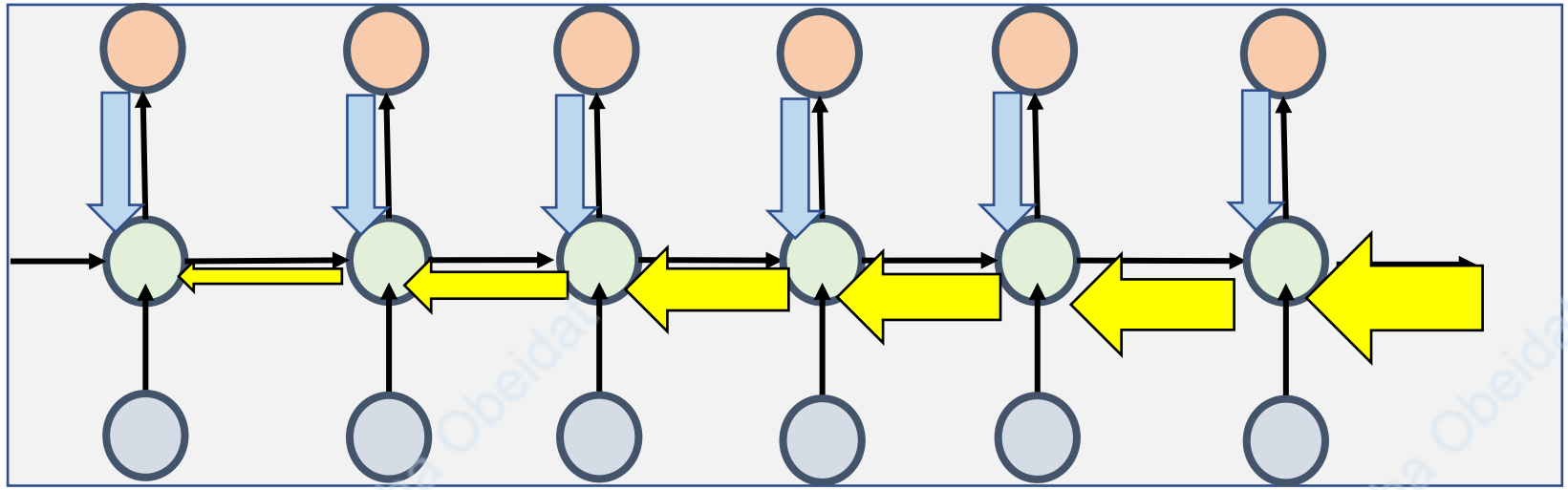
What D has eigen values = .001, $\Rightarrow D^{\mathrm{k}}$ will have 10^{-8} → (vanishing gradient)

Vanishing and exploding gradient

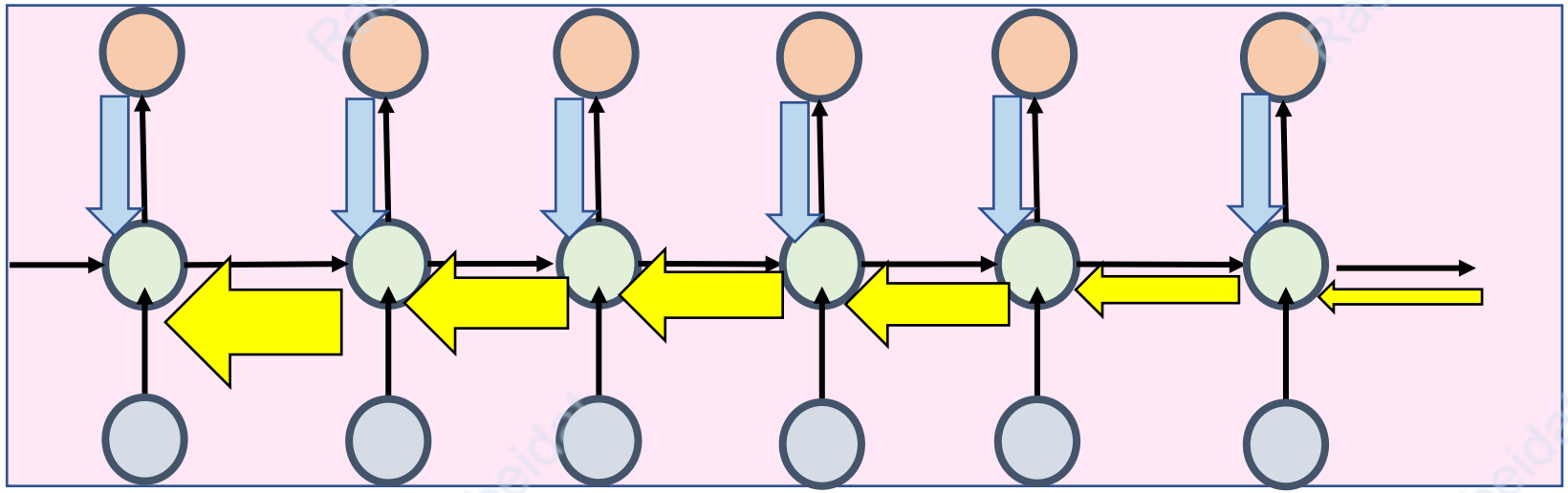
- The gradient grows exponentially or decay exponentially as the number of layers (the length of the sequence) increases. Vanishing gradient.
- If the gradient ***vanishes***, then the earlier hidden states have no real effect on the later hidden states, meaning no long-term dependencies are learned! This result RNN not trained properly.
- If the gradient ***explodes***, this results in large updates to the network weights, and in turn, an unstable network. At extreme cases, the values weights can overflow and result in NaN values.

Vanishing and Exploding Gradient

Vanishing gradient



Exploding gradient



Solution to Vanishing and exploding gradient

- Because of the vanishing gradient problem, its is hard for RNN to carry information from earlier time steps to later ones. In other words, RNN has a **short-term memory**.
- ***LSTM*** and ***GRU*** special kinds of RNN, capable of learning long-term dependencies (Solution to vanishing gradient).
- LSTM and GRU use an internal mechanisms called ***gates*** that can regulate the flow of information and mitigate the problem of vanishing gradients.
- Solution to exploding gradient: ***clipping gradients*** if their norm exceeds a given threshold

Motivating Example: Language Model

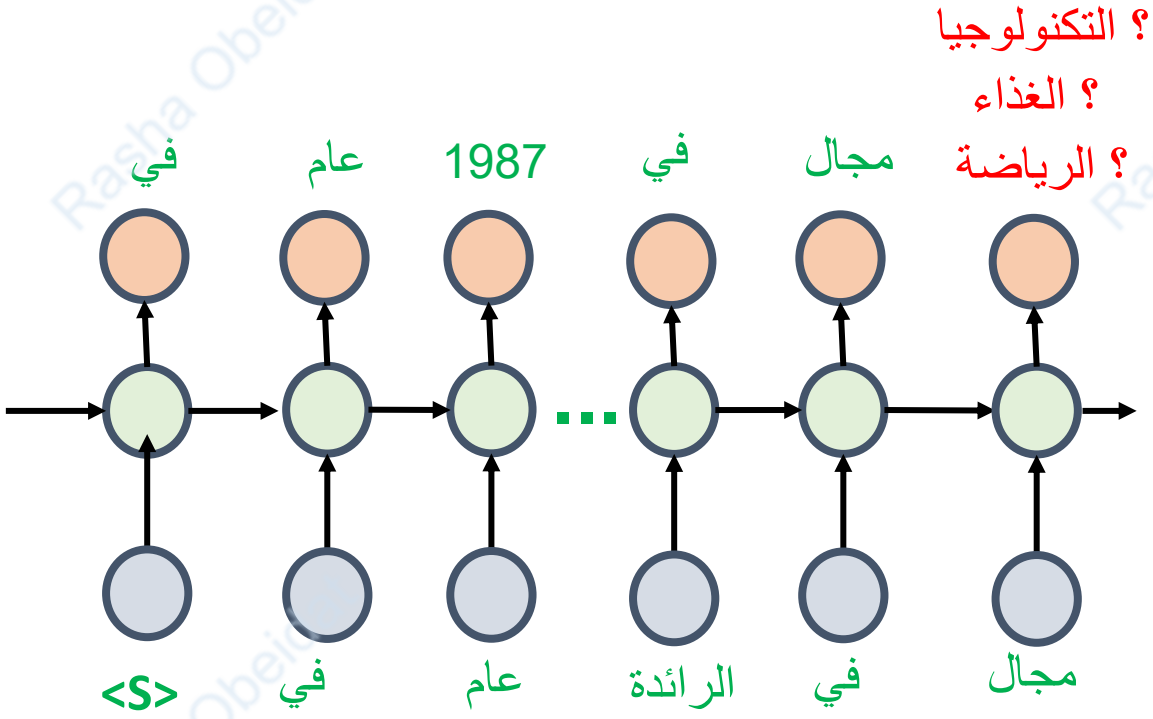
In **Language Models**, the goal is to compute the probability of the **next word** in the sequence.

في عام 1987، أطلقت شركة أبل أول جهاز كمبيوتر محمول لها وقد لاقت منتجاتها اقبالا واسعا. ومنذ ذلك الحين، استمرت الشركة في تطوير منتجاتها. واليوم، تُعد أبل من بين الشركات الرائدة في مجال.....

P(التكنولوجيا | في عام... الرائدة في مجال) ؟

P(الغذاء | في عام... الرائدة في مجال) ؟

P(الرياضة | في عام... الرائدة في مجال) ؟



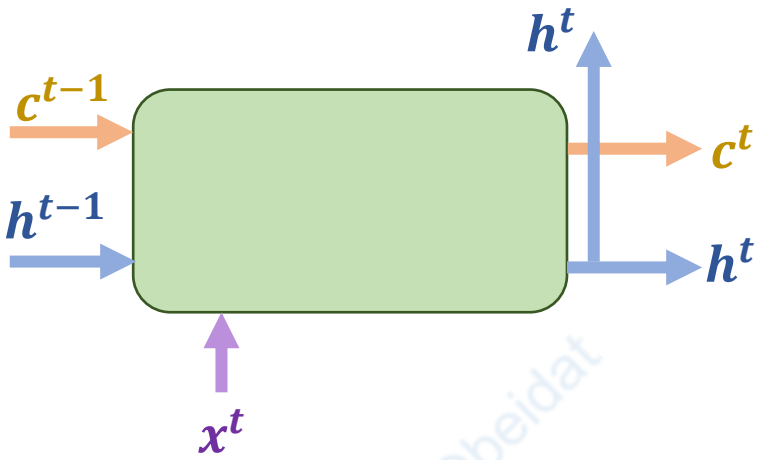
- To correctly assign high probability to "**التكنولوجيا**" the model needs to **remember** " **جهاز كمبيوتر**" which appeared **much earlier** in the sequence.
- However, **vanilla RNNs** suffer from the **vanishing gradient problem**, making them **ineffective at capturing such long-distance dependencies**. As a result, the connection between "**التكنولوجيا**" and " **جهاز كمبيوتر**" might be lost, leading to incorrect predictions.

LSTM: Long Short Term Memory networks

- ***Long Short Term Memory networks (LSTM)*** are a special kind of RNN specifically designed to address the vanishing gradient problem.
- Capable of learning long-term dependencies.
- Achieved state of the art performance on large variety of tasks including: language modeling, speech recognition, and machine translation, protein structure prediction among others.
- As apposite to RNN that has volatile memory, LSTM incorporates ***memory units*** that enable learning complex and long-term dependencies.

LSTM

- Each LSTM unit consists of a recurrent ***memory cell*** and three multiplicative ***gates: input***, ***output*** and ***forget*** gates.
- **The input** to the LSTM are the current input x^t, the previous hidden state h^{t-1}, and the previous memory cell status c^{t-1}.
- **The output** are the current hidden state h^t, and the current memory cell status c^t.



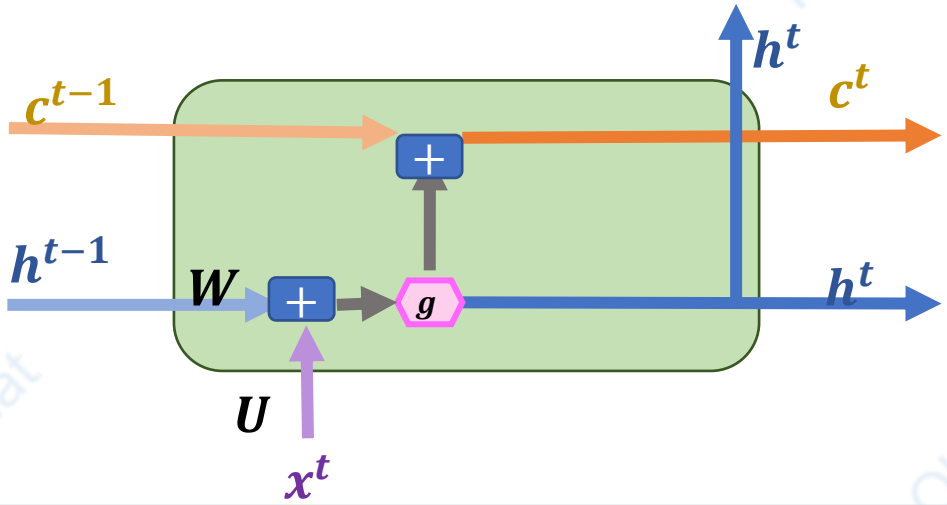
LSTM Cell with No Gates

- Let us start with LSTM cell with no gates and add the gates incrementally.
- Instead of only computing the hidden state h^t at each time step using:

$$h^t = tanh(W_h h^{t-1} + U_h x^t + b_h)$$

- We also can compute the memory cell c^t that has same dimension of the vector h^t as follows:

$$c^t = c^{t-1} + h^t$$



Symbol Key

+ Element-wise summation

g tanh

Adding Gates.

- The gates are used to modulate input and output and decide how much the current cell should remember from the previous history.
- The formula they use consists of non-linearity over the linear combination.
- *Gates* only use the sigmoid for non-linearity. Sigmoid is important so the gate it gives values from zero to one. The extreme value 1 can be interpreted as ”*open the gate*” and the value 0 as a “*close the gate*”.
- Based on the value of the gate vector, We either get the same value from previous time step or zero or something in between.
- All gates are being trained.

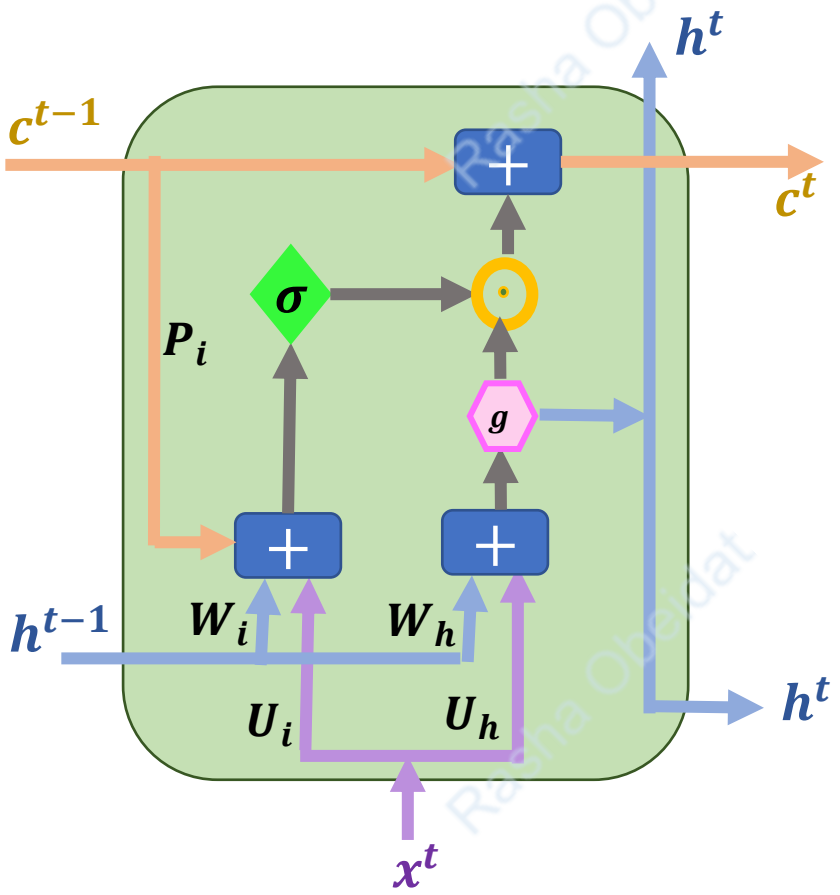
Input Gate

- Some input might be noise and should not affect our memory.
- The ***input gate*** i^t Controls how much of the new information (from the current input x^t) and the hidden state from the previous time step h^{t-1} should be added to (store in) the memory cell c^t.
- Input gate i^t has the same dimensions as the hidden units h^t
- Selecting which input to store in the memory is dependent on:
  - Previous hidden state
  - Current input
  - (optionally on) Previous memory

$$h^t = tanh(W_h h^{t-1} + U_h x^t + b_h)$$

$$i^t = \sigma(W_i h^{t-1} + U_i x^t + P_i \odot c^{t-1} + b_i)$$

$$c^t = i^t \odot h^t + c^{t-1}$$



Symbol Key

- Element-wise summation
- Element-wise multiplication
- σ Sigmoid
- g tanh

Output Gate

The output gate o^t controls what to read from the memory c^t and output to the next hidden state (i.e., h^t)

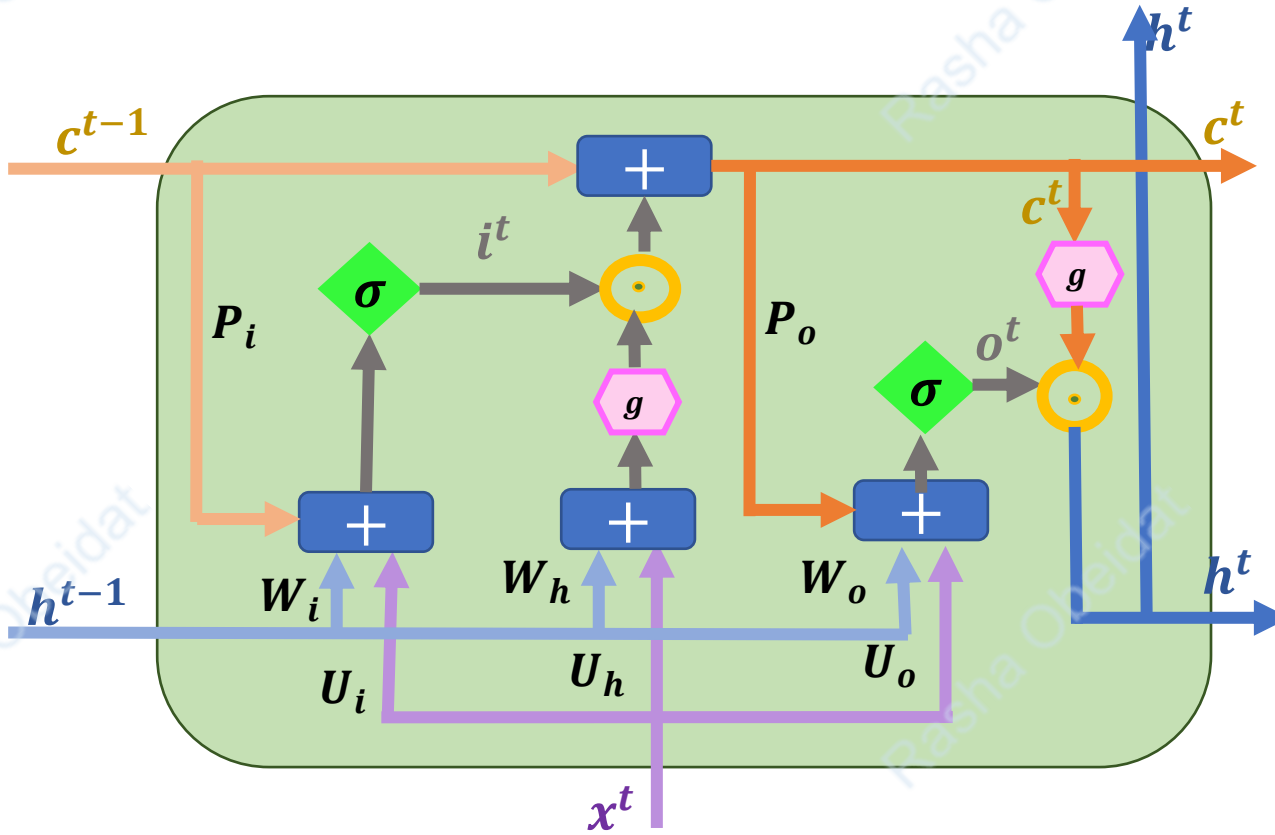
$$z^t = tanh(W_h h^{t-1} + U_h x^t + b_h)$$

$$i^t = \sigma(W_i h^{t-1} + U_i x^t + P_i \odot c^{t-1} + b_i)$$

$$c^t = c^{t-1} + i^t \odot z^t$$

$$o^t = \sigma(W_o h^{t-1} + U_o x^t + P_o \odot c^t + b_o)$$

$$h^t = o^t \odot tanh(c^t)$$



Symbol Key

- Element-wise summation
- Element-wise multiplication
- σ Sigmoid
- g tanh

What about vanishing gradient now?!

- Till this point, adding a memory cell completely solve vanishing gradient problem, why?
- Given that $\boldsymbol{c^t = i^t \odot z^t + c^{t-1}}$, What is the value of $\frac{\partial c^t}{\partial c^{t-1}}$?
- But we can't erase anything from c^{t-1}. What if we want to work with everything sequences? Can c remember everything? Is it a good idea to remember everything?

Forget Gate

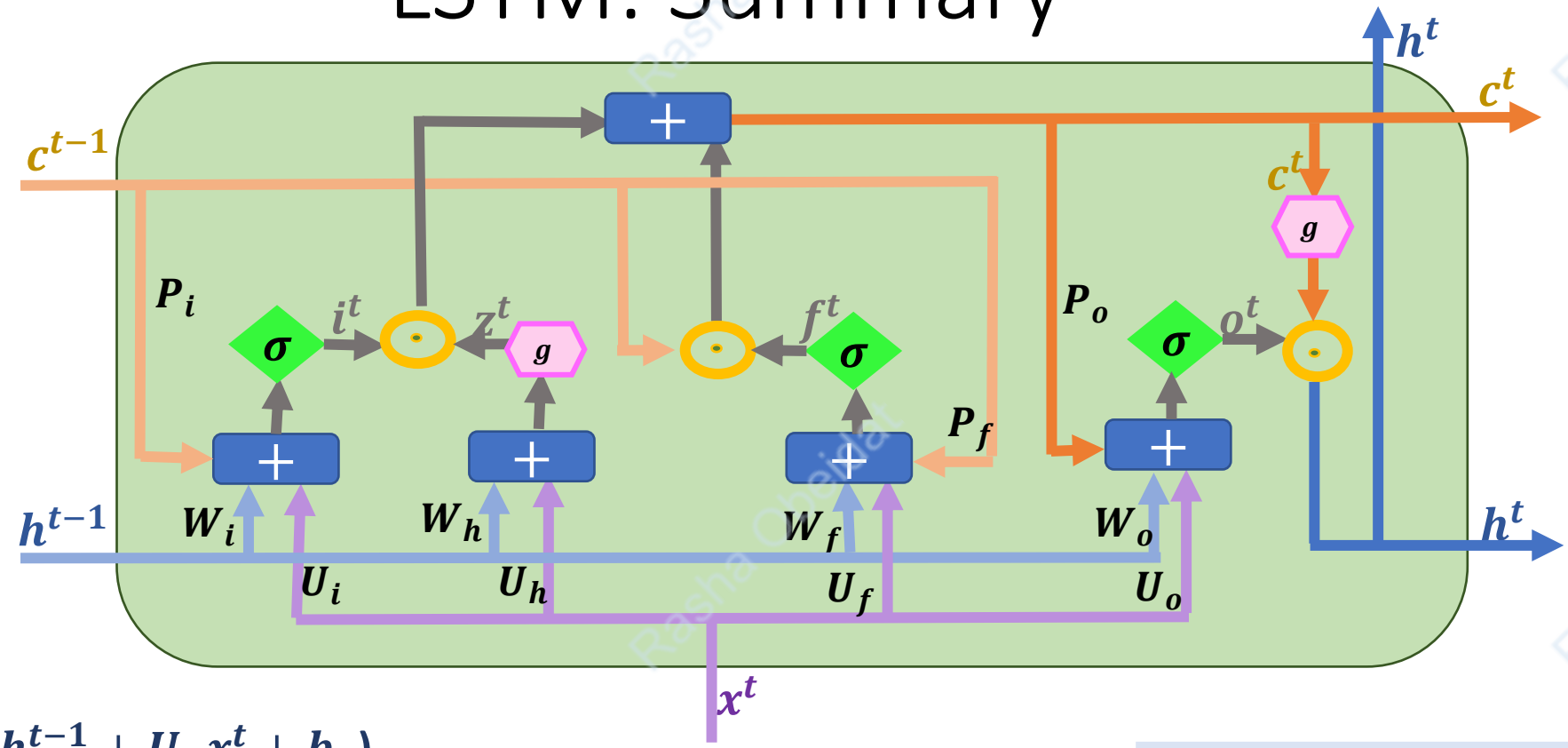
- Memory cell c have a limited capacity which might a lot of time step filled with a lot of not important information.
- LSTM should be able to erase the irrelevant information from the memory sometimes.
- The **forget gate** will help LSTM to achieve that, it decide how much of should we forget.

How much we forget depends

- Previous hidden state $\boldsymbol{h^{t-1}}$
- Current input $\boldsymbol{x^t}$
- Previous memory $\boldsymbol{c^{t-1}}$

$$\boldsymbol{f^t = \sigma(W_f h^{t-1} + U_f x^t + P_f \odot c^{t-1} + b_f)}$$

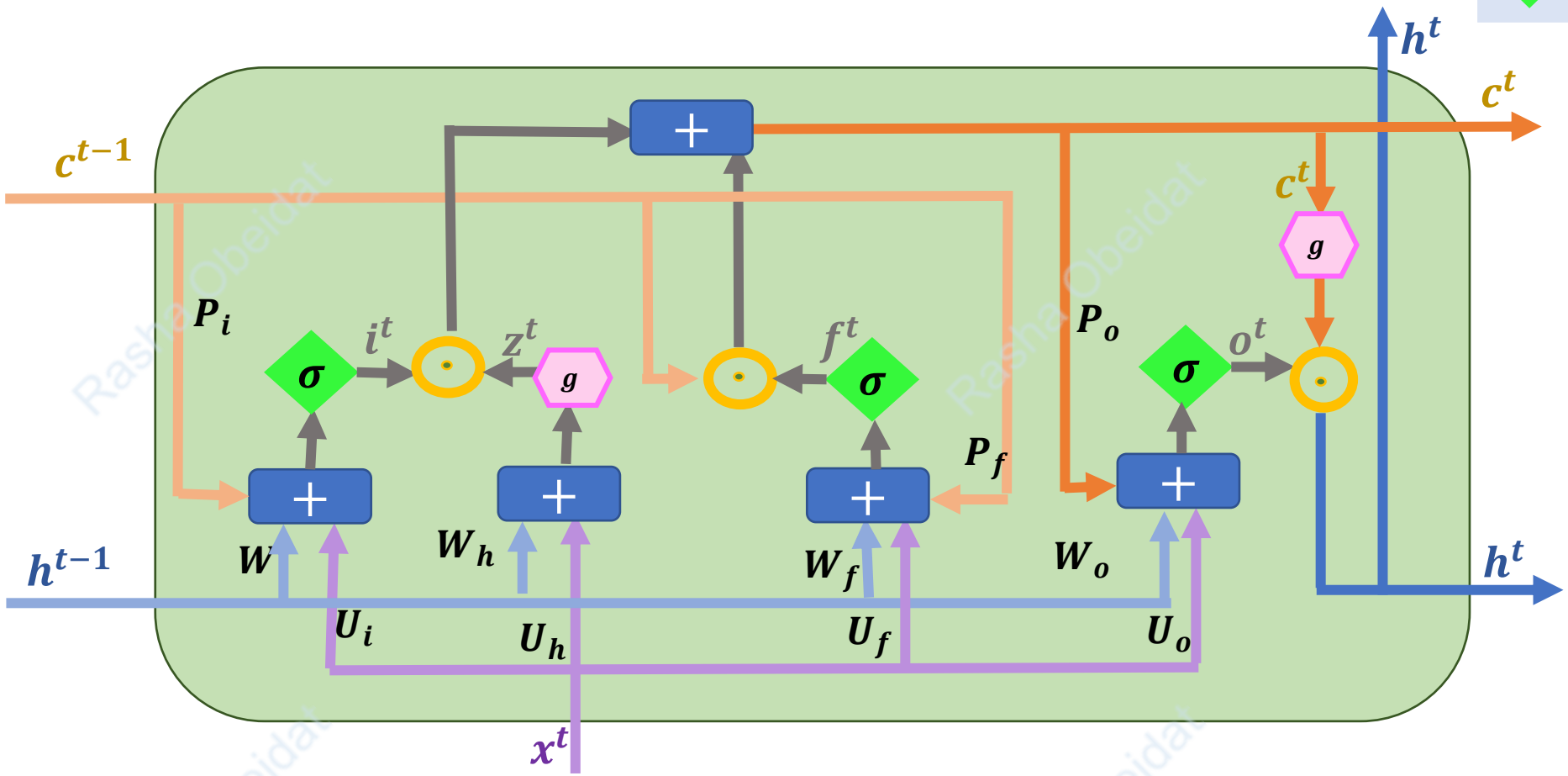
LSTM: Summary



$$z^t = tanh(W_h h^{t-1} + U_h x^t + b_h)$$
$$i^t = \sigma(W_i h^{t-1} + U_i x^t + P_i \odot c^{t-1} + b_i)$$
$$f^t = \sigma(W_f h^{t-1} + U_f x^t + P_f \odot c^{t-1} + b_f)$$
$$c^t = i^t \odot z^t + f^t \odot c^{t-1}$$
$$o^t = \sigma(W_o h^{t-1} + U_o x^t + P_o \odot c^t + b_o)$$
$$h^t = o^t \odot tanh(c^t)$$

Symbol Key

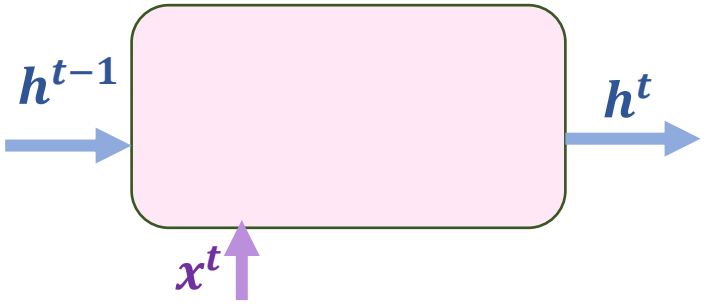
- Element-wise summation
- Element-wise multiplication
- σ Sigmoid
- g tanh



GRU: Gated Recurrent Unit

- Aims to solve the **vanishing gradient problem** which happens with vanilla recurrent neural network.
- GRU's doesn't use memory unit(cell state). Instead, Combines the functionality of the cell state (c^t) in LSTM and hidden state (h^t) into a single state:

  The hidden state (h^t) acts as the sole memory for both long- and short-term dependencies.
- GRU relies on its **update gate** (z^t) and **reset gate** (r^t) to control the flow of information, without needing a separate memory cell.
- Computationally more ***efficient*** to train than LSTM because it is less complex.
- **The input** to the GRU are the current input x^t and the previous hidden state h^{t-1}.
- **The output** are the current hidden state h^t.



Update and Reset Gates.

Update gate (z^t)

- Helps the model determine how much of the past information (h^{t-1}) should be retained and passed forward to the current hidden state h^t (and consequently to the future steps).
- Combines the functionality of the input gate and forget gate from the LSTM into a single gate.
- By controlling the balance between preserving previous hidden state values and incorporating new information to focus on both long-term and short-term

$$z^t = \sigma(U_z x^t + W_z h^{t-1} + b_z)$$

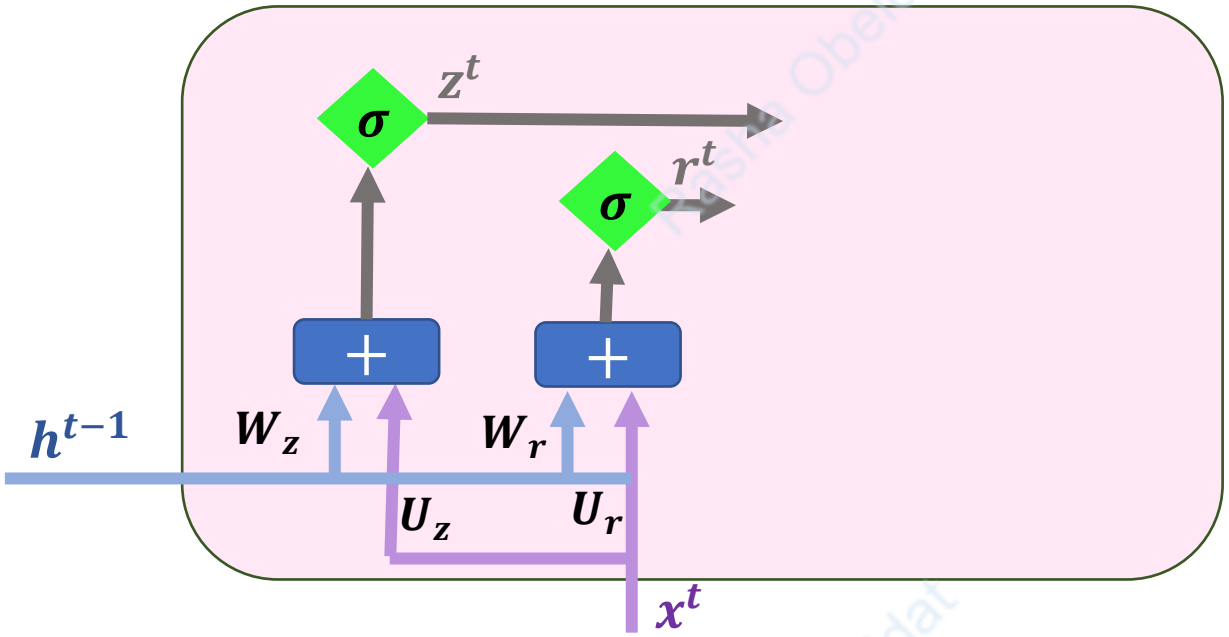
- **Reset Gate** helps the model to decide how much of the past information to forget.

$$r^t = \sigma(U_r x^t + W_r h^{t-1} + b_r)$$

- **Update gate** and **reset Gate** are two vectors have the same dimension as h.

Update and Reset Gates.

- Adding the update gate z^t and the reset gate r^t to the cell



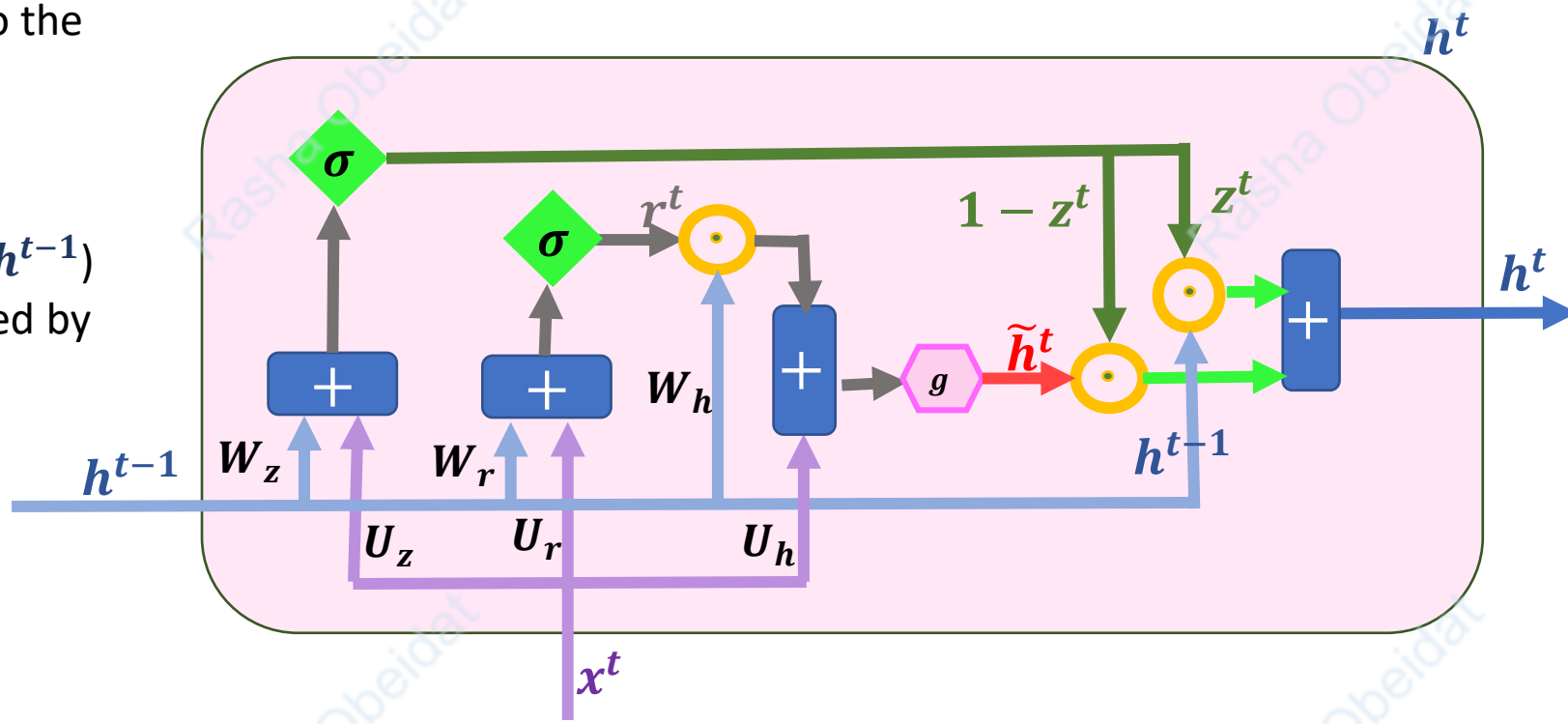
GRU: Summary

$$z^t = \sigma(U_z x^t + W_z h^{t-1} + b_z)$$
$$r^t = \sigma\left(U_r x^t + W_r h^{t-1} + b_r\right)$$
$$\tilde{h}^t = tanh(U_h x^t + r^t \odot W_h h^{t-1})$$
$$h^t = (1 - z^t) \odot \tilde{h}^t + z^t \odot h^{t-1}$$

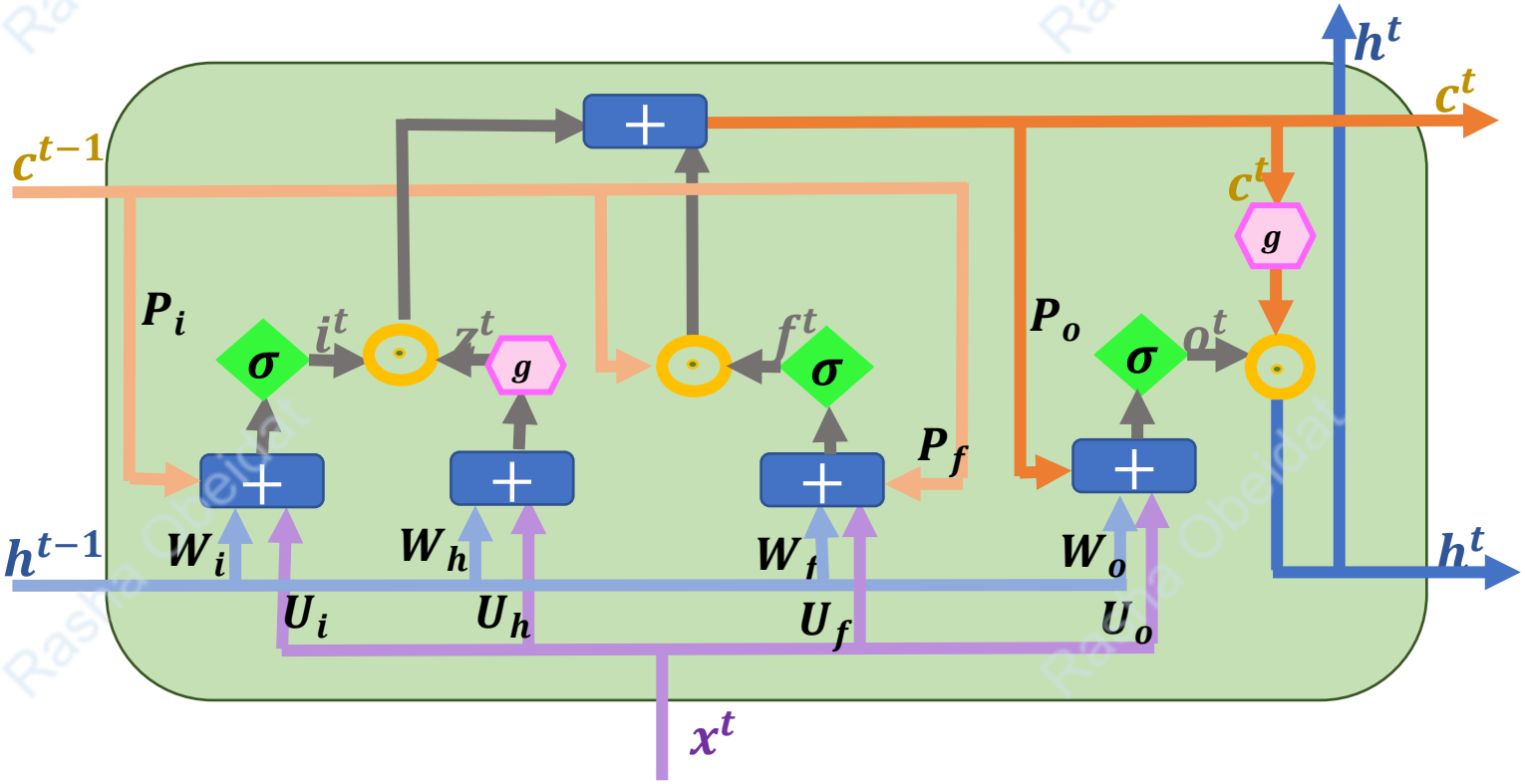
Candidate Hidden State ($\tilde{h}^t$)

- Represents the new memory content to be added to the hidden state.
- The reset gate (r^t) determines how much of the previous hidden state (h^{t-1}) contributes to the candidate.

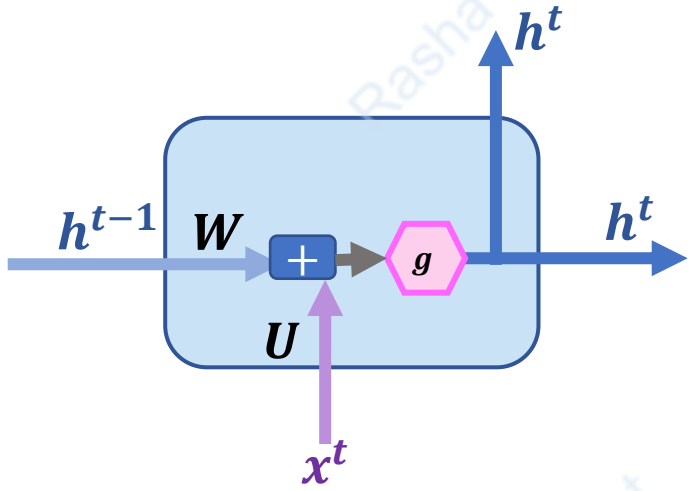
Final Hidden State (h^t):
A combination of the previous hidden state (h^{t-1}) and the candidate hidden state ($\tilde{h}^t$), controlled by the update gate (z^t).



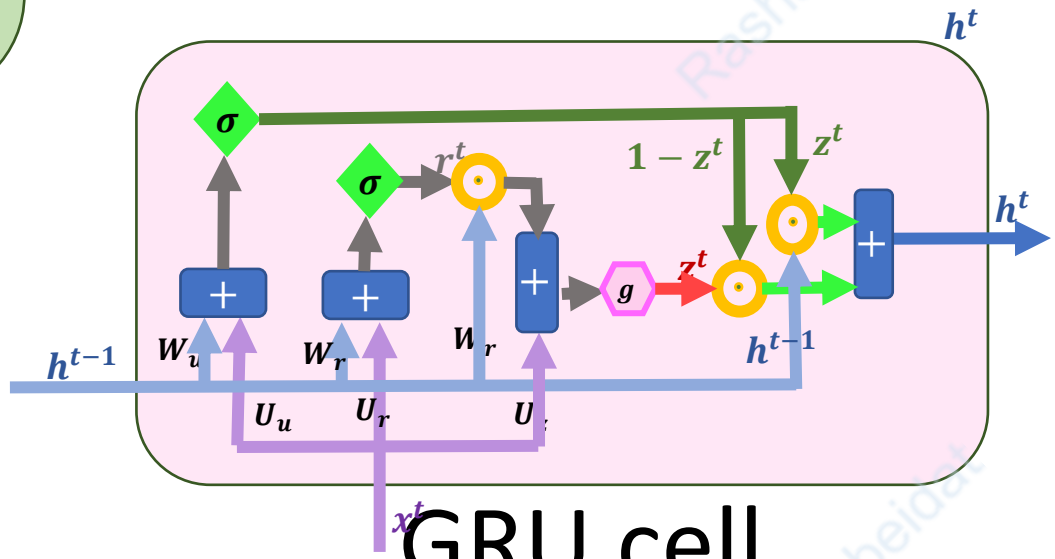
LSTM cell vs GRE cell vs RNN cell



LSTM cell



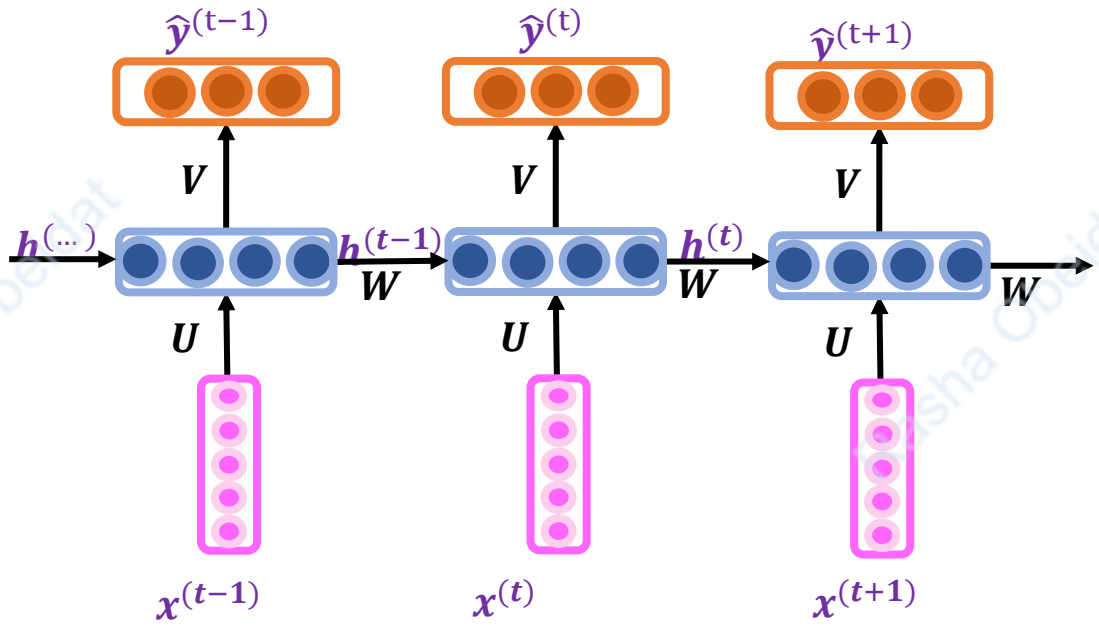
RNN cell



GRU cell

Bi-directional RNN: Motivating example

- One limitation of RNNs (i.e., RNN, LSTM, GRU) is inability to fully capture **contextual dependencies** from both past (*preceding*) and future (*succeeding*) directions in a sequence.
- Solution ? Bidirectional Architecture



Bi-directional RNN: Motivating example

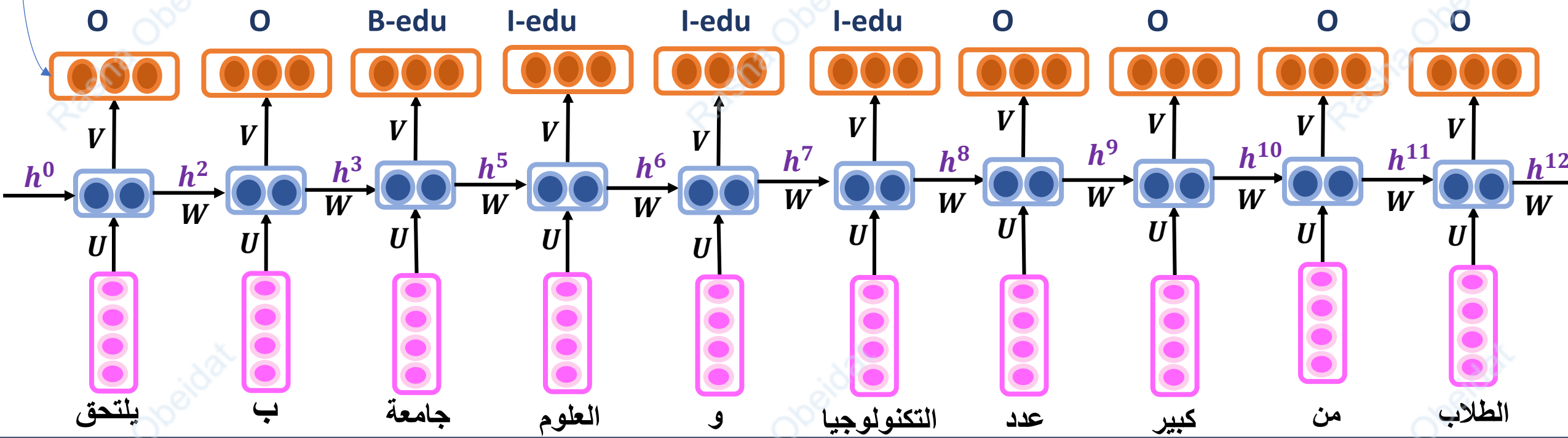
- Named Entity Recognition: the task of identifying and classify **named entity** mentioned in a text into pre-defined categories such as person, educational-institution, organizations, locations.
- For example:
  - يلتحق (بجامعة العلوم والتكنولوجيا Edu-institution) عدد كبير من الطلاب سنويا
- Use ***BIO notation***: tags either
  - Begin-of-entity (B_ET such as B_edu)
  - Or continuation-of-entity (I_ET such as I_edu))
  - Or not entities (O) :

  $Y = \{O, B\text{-}Per, I\_Per, B\text{-}edu, I\_edu, B\text{-}edu, I\_loc, ...\}$,

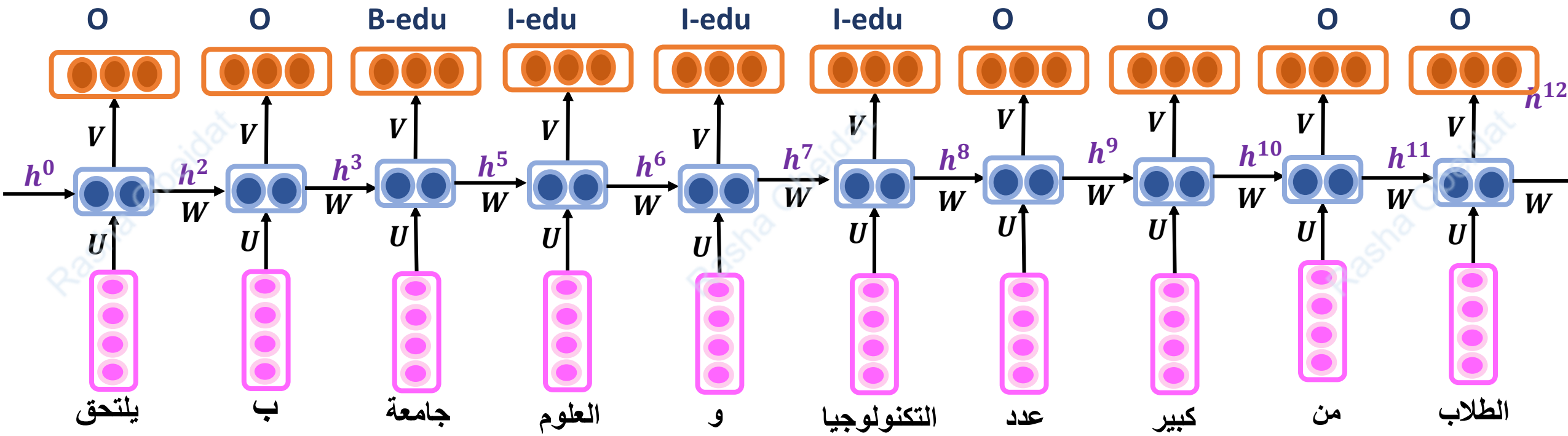
يلتحق	ب	جامعة	العلوم	و	التكنولوجيا	عدد	كبير	من	الطلاب	سنويا
O	O	B-edu	I-edu	I-edu	I-edu	O	O	O	O	O

Bi-directional RNN: Motivating example

- This layer is a softmax that produces a probability distribution over the possible tags (i.e., Y). The softmax function turns "scores" that comes from multiplying V by h^t into a probabilities.



Bi-directional RNN: Motivating example

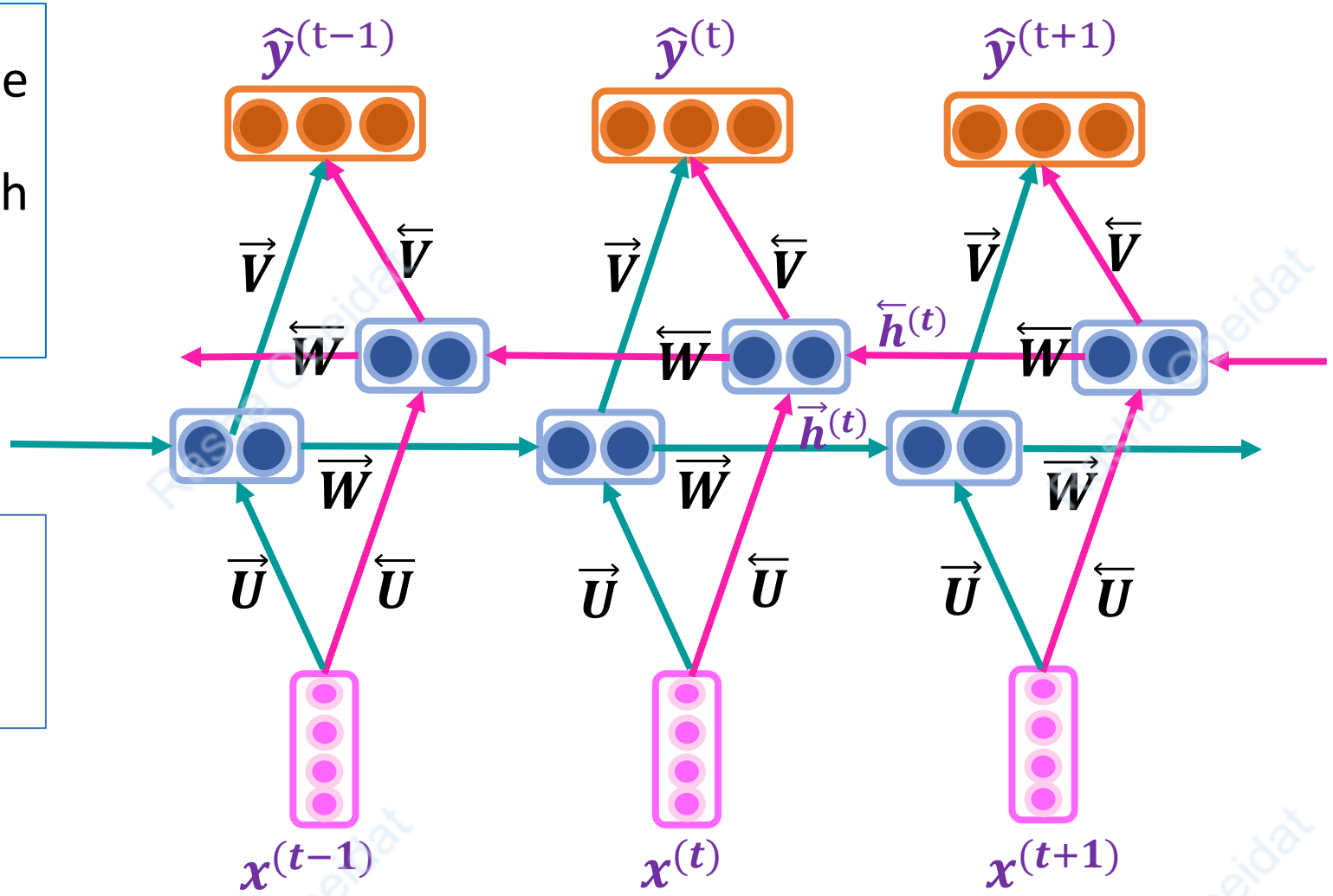


- The word (الطلاب) could be very important to be captured to correctly tag (جامعة العلوم والتكنولوجيا), Standard RNN can't incorporate information words following the named entity.

Bi-directional RNN

Bi-directional RNN incorporate information from words both directions of the sequence.

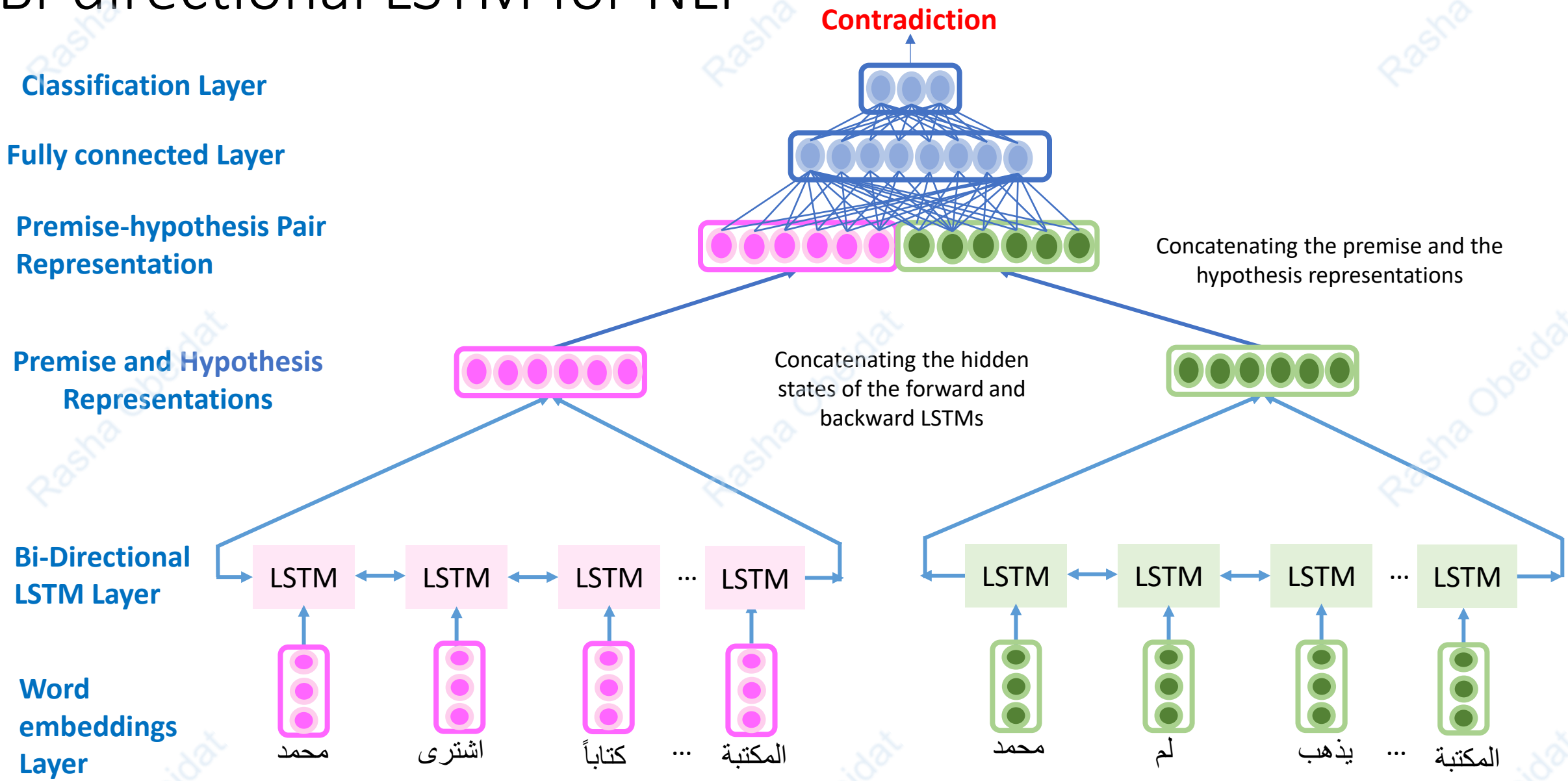
$$\overrightarrow{h}^{(t)} = tanh(\overrightarrow{W}\overrightarrow{h}^{(t-1)} + \overrightarrow{U}x^{(t)} + \overrightarrow{b}_i)$$
$$\overleftarrow{h}^{(t)} = tanh(\overleftarrow{W}\overleftarrow{h}^{(t-1)} + \overleftarrow{U}x^{(t)} + \overleftarrow{b}_i)$$
$$\hat{y}^{(t)} = g(\overleftarrow{V}\overleftarrow{h}^{(t)} + \overrightarrow{V}\overrightarrow{h}^{(t)} + b)$$



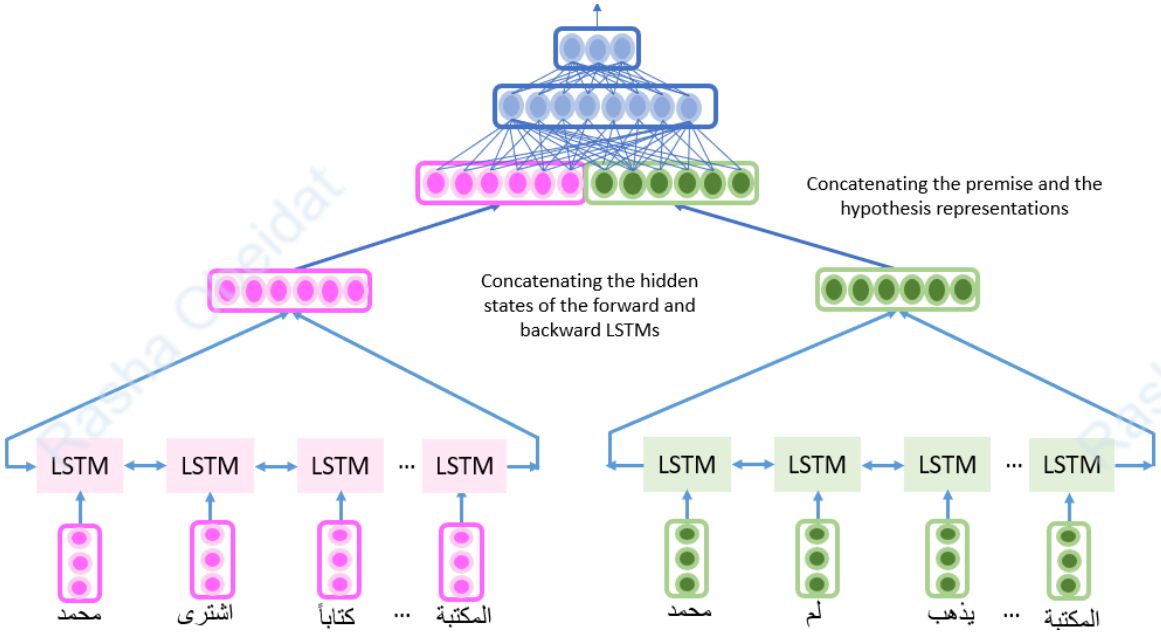
Bi-directional LSTM for NLI

- **Natural Language Inference (NLI)** is a task in NLP that involves determining the logical relationship between two sentences:
  - **Premise**: The first sentence, which serves as the context or basis.
  - **Hypothesis**: The second sentence, which needs to be evaluated in relation to the premise.
- The possible relationships (classes) are:
  - **Entailment**: The hypothesis is true based on the premise.
  - **Contradiction**: The hypothesis is false and contradicts the premise.
  - **Neutral**: The hypothesis cannot be confirmed or denied based on the premise.
- **Example:**
  - **Premise :**"محمد اشترى كتاباً عن البرمجة من المكتبة."
  - **Hypothesis:**
    - **Entailment:** محمد يمتلك الآن كتاباً عن البرمجة
    - **Contradiction:** محمد لم يذهب إلى المكتبة
    - **Neutral:** محمد اشترى أدوات مكتبية أيضاً
- The **input** is *premise-hypothesis* pair and the **output** is the *class (e.g.,* **Contradiction),**

Bi-directional LSTM for NLI



Architecture To Code




```
        embeddings[token] if token in embeddings else np.zeros(embedding_dim)
        for token in tokens[:max_len]
    ]
    while len(vectors) < max_len:
        vectors.append(np.zeros(embedding_dim))
    return np.array(vectors)

# Bi-LSTM-based model
class BiLSTMTextualEntailmentModel(nn.Module):
    def __init__(self, embedding_dim, hidden_dim, output_dim):
        super(BiLSTMTextualEntailmentModel, self).__init__()
        self.lstm = nn.LSTM(embedding_dim, hidden_dim, batch_first=True, bidirectional=True)
        self.fc_hidden = nn.Linear(4 * hidden_dim, 50)  # 2 * hidden_dim for each LSTM (forward and backward)
        self.fc_out = nn.Linear(50, output_dim)
        self.relu = nn.ReLU()

    def forward(self, premise, hypothesis):
        _, (premise_hidden, _) = self.lstm(premise)
        _, (hypothesis_hidden, _) = self.lstm(hypothesis)

        # Concatenate last forward and backward hidden states for both premise and hypothesis
        premise_rep = torch.cat((premise_hidden[-2], premise_hidden[-1]), dim=1)
        hypothesis_rep = torch.cat((hypothesis_hidden[-2], hypothesis_hidden[-1]), dim=1)

        combined = torch.cat((premise_rep, hypothesis_rep), dim=1)
        hidden_output = self.relu(self.fc_hidden(combined))
        output = self.fc_out(hidden_output)
        return output

# Training function
def train(model, train_loader, criterion, optimizer, device):
    model.train()
    total_train_loss = 0
    correct_train = 0
    total_train = 0

    for premise, hypothesis, labels in train_loader:
        premise, hypothesis, labels = premise.to(device), hypothesis.to(device), labels.to(device)

        optimizer.zero_grad()  # Clear previous gradients
        outputs = model(premise, hypothesis)  # Forward pass

        loss = criterion(outputs, labels)  # Compute loss
        loss.backward()  # Backward pass
        optimizer.step()  # Update weights
```

Functions for Loading Word Embeddings

```python
import torch
import torch.nn as nn
from torch.utils.data import DataLoader
import numpy as np
import torch.optim as optim

# Load GloVe embeddings
def load_glove_embeddings(glove_path, embedding_dim):
    embeddings = {}
    with open(glove_path, "r", encoding="utf8") as f:
        for line in f:
            parts = line.strip().split()
            word = parts[0]
            vector = np.array(parts[1:], dtype=np.float32)
            embeddings[word] = vector
    return embeddings

# Determine maximum sequence length
def get_max_length(file_paths):
    max_len = 0
    for file_path in file_paths:
        with open(file_path, "r", encoding="utf8") as f:
            for line in f:
                premise, hypothesis, _ = line.strip().split("\t")
                max_len = max(max_len, len(premise.split()), len(hypothesis.split()))
    return max_len
```

Functions for Loading the Dataset and the Word Embeddings

```python
# Function to load the dataset
def load_dataset(file_path, embeddings, embedding_dim, max_len):
    data = []
    label_map = {"entailment": 0, "contradiction": 1, "neutral": 2}

    with open(file_path, "r", encoding="utf8") as f:
        for line in f:
            premise, hypothesis, label = line.strip().split("\t")
            premise_vec = text_to_embedding(premise, embeddings, embedding_dim, max_len)
            hypothesis_vec = text_to_embedding(hypothesis, embeddings, embedding_dim, max_len)
            data.append((torch.tensor(premise_vec, dtype=torch.float32),
                         torch.tensor(hypothesis_vec, dtype=torch.float32),
                         label_map[label]))
    return data

# Helper function to convert text to embeddings
def text_to_embedding(text, embeddings, embedding_dim, max_len):
    tokens = text.split()
    vectors = [
        embeddings[token] if token in embeddings else np.zeros(embedding_dim)
        for token in tokens[:max_len]
    ]
    while len(vectors) < max_len:
        vectors.append(np.zeros(embedding_dim))
    return np.array(vectors)
```

The Bi-directional LSTM Model

```python
# Bi-LSTM-based model
class BiLSTMTextualEntailmentModel(nn.Module):
    def __init__(self, embedding_dim, hidden_dim, output_dim):
        super(BiLSTMTextualEntailmentModel, self).__init__()
        self.lstm = nn.LSTM(embedding_dim, hidden_dim, batch_first=True, bidirectional=True)
        self.fc_hidden = nn.Linear(4 * hidden_dim, 50)  # 2 * hidden_dim for each LSTM (forward and backward)
        self.fc_out = nn.Linear(50, output_dim)
        self.relu = nn.ReLU()

    def forward(self, premise, hypothesis):
        _, (premise_hidden, _) = self.lstm(premise)
        _, (hypothesis_hidden, _) = self.lstm(hypothesis)

        # Concatenate last forward and backward hidden states for both premise and hypothesis
        premise_rep = torch.cat((premise_hidden[-2], premise_hidden[-1]), dim=1)
        hypothesis_rep = torch.cat((hypothesis_hidden[-2], hypothesis_hidden[-1]), dim=1)

        combined = torch.cat((premise_rep, hypothesis_rep), dim=1)
        hidden_output = self.relu(self.fc_hidden(combined))
        output = self.fc_out(hidden_output)
        return output
```

Model Training (one epoch)

```python
def train(model, train_loader, criterion, optimizer, device): #Perform a single epoch of training.

    model.train()
    total_train_loss = 0
    correct_train = 0
    total_train = 0

    for premise, hypothesis, labels in train_loader:
        premise, hypothesis, labels = premise.to(device), hypothesis.to(device), labels.to(device)
        optimizer.zero_grad()  # Clear previous gradients
        outputs = model(premise, hypothesis)  # Forward pass
        loss = criterion(outputs, labels)  # Compute loss
        loss.backward()  # Backward pass
        optimizer.step()  # Update weights
        total_train_loss += loss.item()
        # Accuracy calculation
        predictions = torch.argmax(outputs, dim=1)
        correct_train += (predictions == labels).sum().item()
        total_train += labels.size(0)

    train_loss = total_train_loss / len(train_loader)
    train_accuracy = correct_train / total_train
    return train_loss, train_accuracy
```

Monitoring the Validation performance during Training

```python
def monitor_validation_during_training(model, dev_loader, criterion, device):
    model.eval()
    total_val_loss = 0
    correct_val = 0
    total_val = 0

    with torch.no_grad():
        for premise, hypothesis, labels in dev_loader:
            premise, hypothesis, labels = premise.to(device), hypothesis.to(device), labels.to(device)

            outputs = model(premise, hypothesis)  # Forward pass
            loss = criterion(outputs, labels)  # Compute loss

            total_val_loss += loss.item()

            # Accuracy calculation
            predictions = torch.argmax(outputs, dim=1)
            correct_val += (predictions == labels).sum().item()
            total_val += labels.size(0)

    val_loss = total_val_loss / len(dev_loader)
    val_accuracy = correct_val / total_val

    return val_loss, val_accuracy
```

Model Training and Evaluation Function

```python
# Main training loop
def train_and_evaluate(model, train_loader, dev_loader, test_loader, criterion, optimizer, device, num_epochs):
    for epoch in range(1, num_epochs + 1):
        # Training phase
        train_loss, train_accuracy = train(model, train_loader, criterion, optimizer, device)

        # Validation phase
        val_loss, val_accuracy = monitor_validation_during_training(model, dev_loader, criterion, device)

        # Print epoch statistics
        print(
            f"Epoch {epoch}/{num_epochs}: "
            f"Train Loss: {train_loss:.4f}, Train Accuracy: {train_accuracy:.4f}, "
            f"Validation Loss: {val_loss:.4f}, Validation Accuracy: {val_accuracy:.4f}"
        )

    # Final test evaluation
    test_loss, test_accuracy = test_model(model, test_loader, criterion, device)
    print(f"Test Loss: {test_loss:.4f}, Test Accuracy: {test_accuracy:.4f}")
```

Setting the parameters and loading the datasets and the embeddings

```python
# Parameters
glove_path = "glove.6B.300d.txt"  # Path to GloVe embeddings
embedding_dim = 300
hidden_dim = 128
output_dim = 3
batch_size = 32
num_epochs = 10
learning_rate = 0.001
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Get maximum sequence length
file_paths = ["train.txt", "dev.txt", "test.txt"]
max_len = get_max_length(file_paths)
print(f"Max sequence length: {max_len}")

# Load GloVe embeddings
glove_embeddings = load_glove_embeddings(glove_path, embedding_dim)

# Load datasets
train_data = load_dataset("train.txt", glove_embeddings, embedding_dim, max_len)
dev_data = load_dataset("dev.txt", glove_embeddings, embedding_dim, max_len)
test_data = load_dataset("test.txt", glove_embeddings, embedding_dim, max_len)
```

Model Training and Evaluation

```
# Create DataLoaders
train_loader = DataLoader(train_data, batch_size=batch_size, shuffle=True)
dev_loader = DataLoader(dev_data, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_data, batch_size=batch_size, shuffle=False)

# Initialize model, criterion, and optimizer
model = BiLSTMTextualEntailmentModel(embedding_dim, hidden_dim, output_dim).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)

# Training, validation, and testing
train_and_evaluate(
    model=model,
    train_loader=train_loader,
    dev_loader=dev_loader,
    test_loader=test_loader,
    criterion=criterion,
    optimizer=optimizer,
    device=device,
    num_epochs=num_epochs
)
```

- **Contact:**
  - Obeidat.rm@gmail.com
  - Rmobeidat@just.edu.jo